HybridCPU ISE v2: A staged-materialization model for SMT-VLIW execution with runtime-owned legality and invalidation-driven replay boundaries

Yuriy A.K.
Independent Researcher, Moscow, Russia
yuriyyak@gmail.com

Artifact DOI: 10.5281/zenodo.20137220 **Repository:** github.com/yuriyyak23/HybridCPU-v2

Abstract

HybridCPU ISE studies a bounded bundle-first execution model for typed SMT/VLIW-style packing over a live PRF/rename-backed backend substrate. The model exposes a canonical decoded bundle at ingress, maps execution to a fixed typed `W=8` lane substrate, and keeps final admissibility at runtime rather than assigning correctness authority to compiler-emitted packet facts. Its central contour separates class-level admission from concrete lane materialization: runtime legality first decides whether a candidate may consume residual typed capacity under boundary, owner/domain, replay, witness, and pressure state; a later materialization step selects a placement-feasible lane inside the admitted class envelope. The same legality world governs a bounded assist plane whose admission/rejection policy is exercised by targeted diagnostics; post-admission discard and non-retirement are treated as code-backed properties with explicit test-local lifecycle evidence rather than as a complete production-retire proof. Replay reuse is likewise bounded: phase keys, structural identity, bundle metadata, boundary state, class templates, invalidation reasons, and replay tokens define reuse and rollback surfaces without claiming global hidden-state determinism or backend-global rewind. The implementation-backed evidence is therefore intentionally scoped: it supports analyzable typed-bundle composition, legality provenance, assist admission governance, replay-template reuse with invalidation and live-witness fallback, and bounded retire publication, but not timing, area, PPA superiority, official SPEC CPU equivalence, or a complete precise-exception theorem. The paper should therefore be read as an implementation-backed architecture report, not as a silicon, PPA, benchmark-superiority, or security-proof paper.

1 Introduction

Contemporary CPU design is often framed as a choice between hidden-window out-of-order execution and compiler-closed packet execution [1, 3, 5, 8, 9, 23]. The current HybridCPU mainline instead studies a bundle-first execution protocol over a live PRF/rename-backed backend substrate. It exposes bundle structure at canonical ingress, executes on a fixed typed `W=8` substrate, keeps legality authority at runtime, admits residual work through a typed-slot admission/materialization path, couples auxiliary data movement to the same bundle model, and bounds replay reuse by explicit invalidation rules.

The execution fabric is heterogeneous and fixed. Four ALU lanes, two LSU lanes, one DMA/stream lane, and one aliased branch/system lane form a class-capacity vector rather than a scalar issue width. Residual capacity is therefore not "any empty slot." It is typed residual capacity whose reuse depends on

class occupancy, ownership and domain guards, legality state, replay state, and dynamic pressure in the current working bundle, a setting adjacent to but more constrained than conventional SMT bandwidth sharing [4, 21].

The empirical motivation is now broader than smoke references or toy runtime matrices. In addition to proof and mechanism-facing runtime artifacts, the paper draws on a long-run SPEC-like diagnostic harness in `TestAssemblerConsoleApps` that repeats architecturally shaped reference slices at large iteration budgets. This evidence is intentionally described as SPEC-like rather than official SPEC CPU, but it matters because it shows that typed-slot legality, replay, and wide packed execution remain measurable under long dynamic runs rather than only in short demonstrations.

The compiler/runtime boundary is explicit but staged. The ingress/version handshake is versioned and fail-closed. Inside a valid contract version, however, missing typed-slot facts remain compatible in the current mainline, while present facts remain structured handoff, validation, and quarantine metadata rather than the present admission authority. Final admissibility remains runtime-owned because the decisive conditions include live bundle state, boundary state, replay context, and dynamic pressure that are not closed at compile time.

This explicit execution law also does not eliminate backend state management. The live machine retains a PRF/rename-backed backend substrate for speculative naming, committed naming, publication, and rollback. Typed placement therefore constrains bundle composition without implying a rename-free backend or a front-end-only correctness story.

The repository name is historical. In this paper, `HybridCPU` names the studied artifact and protocol surface, not a claim that the live machine forms a rename-free third family outside PRF/rename execution.

Replay and assist are likewise bounded. Replay reuse is valid only inside an explicit envelope defined by replay phase identity, structural compatibility, owner/domain continuity, and invalidation rules. Replay tokens restore only selected architectural-register and exact bound main-memory surfaces under owner-thread and side-effect-gating metadata; they do not reinstall a global pre-rollback backend image. Assist work is a governed auxiliary plane: its admission/rejection policy is exercised through residual legality, residual lane capacity, backpressure, quota, owner/domain, invalid-replay, and primary-priority checks, while post-admission discard and non-retirement are treated as code-backed properties with explicit test-local lifecycle evidence.

The contribution claimed here is therefore narrow and falsifiable. The repository exposes a bundle-first execution protocol in which legality, typed placement, assist participation, and replay reuse are first-class execution semantics rather than hidden scheduler side effects.

2 Problem Statement

The problem addressed by HybridCPU is not generic wide-issue utilization. The live architecture must decide, at runtime, whether candidate work may lawfully share a fixed typed `W=8` substrate while four hardware thread contexts, an auxiliary assist plane, and replay-qualified reuse all interact with the same bundle boundary. In such a machine, free physical width is not itself the relevant quantity. The relevant quantity is legal typed residual capacity under current class occupancy, boundary state, ownership/domain scope, legality-witness state, replay phase state, and dynamic pressure, including memory-system pressure that cannot be reduced to static packet shape [2].

This question cannot be closed by the compiler alone. The compiler can form bundles, classify structure, and emit typed-slot facts, but the present mainline stages those facts as validation and agreement evidence rather than as mandatory correctness carriers. More fundamentally, final admissibility depends on live runtime conditions that do not exist at compile time. Runtime legality must therefore remain authoritative even when compiler structure is useful.

The same problem extends beyond foreground issue. Auxiliary work must remain inside the legality model if its coexistence with retire-visible work is to be explained precisely. Replay reuse must remain

bounded by explicit invalidation if previously lawful placement is to be reused without overstating deterministic reproduction. HybridCPU therefore addresses a problem of architectural legibility under dynamic conditions: how to adapt bundle composition at runtime without collapsing legality, assist behavior, and replay into opaque backend behavior or into compiler-only assumptions.

3 Why Existing Architectural Labels Do Not Capture HybridCPU

HybridCPU inherits explicit bundle structure from packet-oriented designs because ingress publishes a canonical decoded bundle and the lane map is fixed and typed. The closest lineage is therefore a VLIW-derived bundle format rather than classical VLIW authority allocation. That label is incomplete, because bundle formation is not the final legality authority. Runtime still decides coexistence, consumes explicit legality results, and performs final lane realization on the typed-slot path.

HybridCPU is adjacent to SMT because four hardware thread contexts share one execution fabric and may cohabit a bundle. That label is also incomplete, because coexistence is not expressed as unrestricted competition for an anonymous issue pool. It is expressed as bundle composition over typed residual capacity, under explicit legality, ownership, and replay constraints.

HybridCPU is likewise not renaming-free. The live backend retains a PRF/rename-backed backend substrate. The architectural claim is therefore not that dynamic backend state has disappeared. The claim is that bundle-local legality and typed placement have been made explicit enough to study as architectural mechanisms while that substrate remains live.

4 Scope, Implementation Boundary, and Claim Envelope

This paper is scoped to the current HybridCPU ISE mainline, with the typed-slot path treated as a live opt-in architectural contour rather than as the sole scheduler contour. Its positive claims are limited to mechanisms that are live today: a fixed typed `W=8` substrate, runtime-owned legality decisions, a typed-slot admission/materialization path with a retained exact-slot compatibility path, assist-coupled auxiliary execution on the same typed substrate, a canonical bundle ingress boundary, a versioned fail-closed compiler/runtime handshake, and replay-stable reuse only while the published replay envelope remains satisfied. The repository also exposes an eight-slot compat transport surface from which a `2048-bit` bundle width may be inferred as a container fact, but the paper does not derive a hardware-cost theorem from that width alone. Likewise, `W=8` is not treated as eight independent useful retiring IPC slots in the current diagnostics; the packed NativeVLIW evaluation is interpreted against its observed effective retire-visible surface. The empirical evidence envelope is likewise bounded: the current long-run harness is SPEC-like repeated-slice evidence, not official SPEC CPU equivalence.

The paper also states several explicit non-claims.

- It does not claim a rename-free backend. The PRF/rename-backed backend substrate remains part of the live machine.
- It does not claim that compiler-emitted typed-slot facts are already mandatory for canonical correctness or admission.
- It does not claim that the scheduler has already collapsed to one universal path. The exact-slot compatibility path remains live as a bounded compatibility contour.
- It does not claim universal hidden-state determinism, universal rollback, or a complete precise-exception theorem.

Table 1: Claim envelope and excluded interpretations.

Mechanism	Positive claim	Explicit non-claim
Typed topology	Fixed typed `W=8` lane map with branch/system aliasing on lane `7`.	Not eight independent scalar or useful IPC lanes, and not a PPA theorem.
Compiler contract	Version handshake is mandatory and fail-closed.	Typed-slot facts are validation-only evidence, not mandatory admission authority.
Legality	Runtime legality service is the ordered authority over guards, replay reuse, and witness fallback.	A witness or compiler fact is not authority by itself.
Admission	Stage A / Stage B defines the opt-in typed-slot contour.	Exact-slot compatibility fallback is live and outside Stage A / Stage B claims.
Replay and assist	Replay reuse and assist validity are invalidation-bounded.	No global determinism, universal rollback, or fully invisible assist claim.
Retire, memory, faults	Retire-local publication and materialized-window fault precedence are bounded mechanisms.	No full memory-model proof and no complete precise-exception theorem.

- It does not claim a fresh repository-wide green total. The checked-in validation story remains a bounded smoke baseline plus named proof and documentation surfaces.
- It does not claim a coequal multi-frontend runtime. Canonical bundle ingress is the active boundary, while compatibility ingress remains quarantined.
- It does not claim timing, area, or PPA superiority. Detailed hardware-cost analysis remains outside the present evidence envelope.
- It does not claim official SPEC CPU equivalence. The long-run evaluation surface is a truthful SPEC-like harness over repeated reference slices.
- It does not claim a global peak IPC of five. The $W_{\text{eff}} \approx 5$ language describes the observed effective retire-visible surface of the current NativeVLIW diagnostic envelope.

These limits are not peripheral caveats. They define the actual falsifiable claim envelope of the present paper.

5 Summary of Contributions

The contributions are mechanism claims tied to the live implementation.

1. The paper defines a bundle-first execution model on a fixed typed `W=8` substrate with explicit lane classes shared across four hardware thread contexts.
2. The paper defines runtime-owned legality authority in which admission consumes explicit legality results rather than treating compiler metadata or raw structural masks as final authority.
3. The paper defines, on the live opt-in typed-slot path, a two-stage mechanism that separates class-level admission from concrete lane materialization and thus makes typed bundle composition explicit without claiming that the retained exact-slot path has disappeared.
4. The paper defines an assist-coupled auxiliary plane with explicit kind, carrier, donor, visibility, quota, backpressure, and admission boundaries, while labeling post-admission discard/non-retirement evidence as test-local.

5. The paper defines a bounded replay envelope in which reuse depends on explicit phase identity, guarded structural compatibility, and invalidation, with rollback support limited to explicitly captured architectural-register, exact bound main-memory, owner-thread, and side-effect-gating state rather than token-only replay.

6. The paper defines a staged compiler/runtime contract in which the ingress/version handshake is fail-closed, while typed-slot facts remain compatibility-tolerant validation evidence inside a valid contract version rather than current correctness authority.

7. The paper defines a bounded evidence story consisting of topology, legality, assist, replay, validation artifacts, and long-run SPEC-like repeated-slice evidence whose implementation anchors are preserved in repository documentation and proof suites.

There is therefore no contradiction between handshake enforcement and fact compatibility: they govern different layers of the current contract.

The remainder of the paper follows that mechanism order: architectural positioning, frontend and bundle representation, legality and replay-qualified witness reuse, two-stage admission, assist-coupled execution, replay boundaries, and finally telemetry, validation, and evaluation.

6 Background and Architectural Positioning

HybridCPU is a typed `W=8` bundle-execution protocol over a shared SMT substrate whose defining boundary is not raw width alone, but runtime authority over legality, typed placement, and execution ownership [4, 21, 22]. At ingress the active machine consumes a canonical bundle format derived from the current ISA-v4 packet encoding. Downstream, a retained exact-slot compatibility path remains live as a bounded compatibility contour, but the main research mechanism is typed bundle composition over a heterogeneous lane map. The implementation is therefore best read as a bundle-first execution protocol above a live PRF/rename-backed backend substrate, with the hybridization limited to the frontend-to-issue contour rather than claimed as a rename-free whole-machine family.

This positioning matters because the repository does not treat a fetched packet as an opaque long word that hardware blindly trusts, nor as a transient input to a generic ready-issue reservoir [1, 5, 8, 23]. Canonical ingress publishes a structured bundle object; decode-side analysis classifies its typed and dependency-bearing structure; later admission remains runtime-owned. The architectural center of gravity therefore lies in explicit legality-bearing bundle structure over a typed lane substrate, even though a narrower compatibility contour still persists.

The retained compat transport density sharpens that positioning. An eight-slot compat bundle occupies `2048` bits, but the importance of that fact is not a hardware-cost theorem and not a claim that raw transport is the active frontend contract. Its importance is architectural: a dense container that colocates execution-control, addressing, vector-length, stream, and transport hints makes legality descriptors, runtime-local structural witnesses, structural identity, and replay-envelope reuse first-class coordination mechanisms rather than cosmetic fast paths. The active contract remains the canonical decoded bundle, not a raw long word.

6.1 Limits of ROB-Centric Out-of-Order Comparisons

The relevant contrast with ROB-centric out-of-order execution is narrower than a generic throughput comparison. In ROB-centric machines, legality, readiness, and placement are largely internal to rename state, issue queues, wakeup-select logic, speculation tracking, and reorder-mediated recovery [1, 8, 9]. HybridCPU addresses a different question: whether packet structure can remain explicit while cross-thread coexistence and residual-capacity reuse are decided at the bundle level rather than hidden inside a fully generic issue pool.

This difference does not imply removal of backend state. The live machine keeps a PRF/rename-

backed backend substrate. What changes is the architectural locus of the bundle-composition decision. Typed-slot legality and placement constrain which additional work may enter a bundle; they do not replace speculative naming, committed naming, or retire publication.

6.2 Why Pure Static VLIW Is Also Inadequate

The corresponding contrast with classical VLIW is equally specific. Static VLIW and EPIC preserve packet structure by moving most packing responsibility to the compiler [5,23,36], but HybridCPU studies cases in which executability is not reducible to offline slot filling. Typed residual capacity may exist in a packet, yet its reuse still depends on boundary state, ownership/domain scope, replay context, and dynamic pressure that are only partially known before execution.

HybridCPU therefore does not treat compile-time packing as the sole correctness authority for wide execution. The decoded bundle is explicit, but executability is staged. Decode and compiler preflight can classify slot class, pinning, register footprints, and local hazard structure; runtime still decides whether the presently formed bundle can admit additional work and how a class-flexible claim is materialized into a concrete lane.

6.3 From Slack-Recovery Lineage to Execution-Policy Architecture

The project has historical lineage in slack-recovery thinking, because partially filled packets make residual capacity visible. That lineage is useful as motivation, but it is no longer the best architecture-level description of the current mainline. The live mechanism is broader: fixed typed execution space, decoded legality descriptors, runtime-owned admission, assist-aware participation, replay-aware reuse, and bounded publication of architectural effects all participate in the same execution law.

The mature formulation is therefore not "slack recovery" in the abstract, but 4-way SMT over a typed `W=8` lane substrate with explicit legality and replay boundaries. Under that formulation, legality is not a post hoc mask check, and packing is not a synonym for opportunistic hole filling. The machine reasons over a structured bundle, admits one more class claim only when residual capacity and legality permit it, and then materializes a placement-feasible lane inside the admitted class envelope.

6.4 Positioning Against OoO SMT, VLIW/EPIC, SIMT, and Adjacent Classes

HybridCPU is adjacent to OoO SMT because multiple hardware threads can contribute to one cycle of execution, but that comparison holds only if SMT is qualified by typed bundle-local legality [4, 21, 22]. Threads do not simply compete for anonymous issue bandwidth; they contribute candidate work to residual typed capacity when legality, ownership, and class constraints permit bundle co-composition.

HybridCPU is adjacent to VLIW and EPIC because packet structure remains explicit and ingress consumes a VLIW-derived bundle format [5, 23, 36]. The divergence is that the packet is not treated as compiler-closed. Canonical ingress publishes a structured bundle object, and runtime remains authoritative for legality, final admission, and lane realization. It is also distinct from dynamic translation systems that form or optimize wider internal packets behind a software or microcode translation boundary [34, 35].

Dynamic heterogeneous-VLIW rescheduling has also been used for fault-tolerant replication, but HybridCPU treats typed residual capacity as a guarded legality/admission surface rather than as a replication-and-voting schedule.

The relation to SIMT, spatial, and stream-oriented designs is different again [12, 13, 24, 30, 40–42]. HybridCPU contains DMA/stream and vector surfaces, but its mainline execution law is not lockstep control over homogeneous lanes. The target setting is heterogeneous general-purpose execution with scalar, memory, control, system, and auxiliary activity sharing one typed packet substrate.

This paper therefore studies a bounded architectural point: canonical bundle ingress, typed residual-

capacity admission, explicit legality provenance, bounded replay reuse, and a live PRF/rename-backed backend substrate. Table 2 and Figure 5 fix that substrate and contour for the rest of the manuscript, while the repository evidence map carries the implementation anchors that support this positioning.

7 Architectural Overview and Frontend Contract

At the frontend boundary, HybridCPU is bundle-first, but the relevant bundle is not a raw long word. The active ingress publishes a canonical decoded bundle, and the runtime reasons about that bundle through typed placement facts, slot-local descriptors, dependency summaries, ownership metadata, and later legality state. The frontend/backend seam is therefore defined by canonical bundle decode into a structured architectural object.

That seam also has a backend side that explicit placement does not replace. The typed-slot admission/materialization path governs lane eligibility, coexistence, and final lane binding, whereas the PRF/rename-backed backend substrate still governs physical value identity, speculative naming, committed naming, retire publication, and rollback surfaces.

Figure 1 fixes this backend boundary visually.

This section fixes six points used later in the paper. First, the only active frontend is canonical bundle ingress derived from the ISA-v4 packet encoding. Second, the execution fabric is a typed `W=8` substrate rather than an anonymous wide-issue pool. Third, the runtime bundle object is a decoded legality carrier rather than an opcode shell. Fourth, compiler-supplied structure is versioned, staged, and validated when present, but it is not the present correctness authority. Fifth, execution ownership is runtime-owned through owner thread, owner context, and domain state rather than through transport metadata alone. Sixth, the paper's later legality, replay, assist, retire, and fault claims are interpreted over a minimal state tuple and named transition surface rather than over prose labels alone.

7.1 Canonical Bundle Ingress and the Active Decode Boundary

The active ingress boundary is the canonical bundle format only. Canonical ingress accepts the current ISA-v4 opcode space, publishes one logical slot per physical slot, and rejects unsupported or quarantined contours at the ingress boundary. Compatibility helpers may persist for diagnostics or historical ingress, but they are not coequal active frontends. The architectural claim is therefore a single active canonical bundle ingress with a frozen compatibility boundary.

This frontend claim is narrower than a claim about the entire scheduler. After decode, the runtime still contains both the typed-slot admission/materialization path and a retained exact-slot compatibility path. The latter is a bounded compatibility contour, not a second decode frontend. Bundle-first therefore names the active ingress contract and the main scheduler contour without denying that a downstream compatibility path remains live.

The retained compat container is nevertheless concrete. Each compat slot serializes to `32` bytes = `256` bits, so a full eight-slot compat bundle occupies `8 x 32 bytes = 256 bytes = 2048 bits`. That width is a transport and container fact only. The active architectural ingress remains the canonical decoded bundle, and the paper does not derive a hardware-cost theorem from compat width alone.

7.1.1 Typed `W=8` Lane Topology

The `W=8` packet is a typed lane vector, not eight interchangeable positions. Table 2 freezes the architectural topology.

Figure 2 gives the same topology as a lane map.

Equivalently, the typed class masks are fixed as eight-bit lane masks

$$M_{\text{alu}} = \texttt{0b00001111},$$
$$M_{\text{lsu}} = \texttt{0b00110000},$$
$$M_{\text{dma}} = \texttt{0b01000000},$$
$$M_{\text{branch}} = M_{\text{system}} = \texttt{0b10000000},$$
$$\text{SharedLane}(\text{branch}, \text{system}) = 7.$$

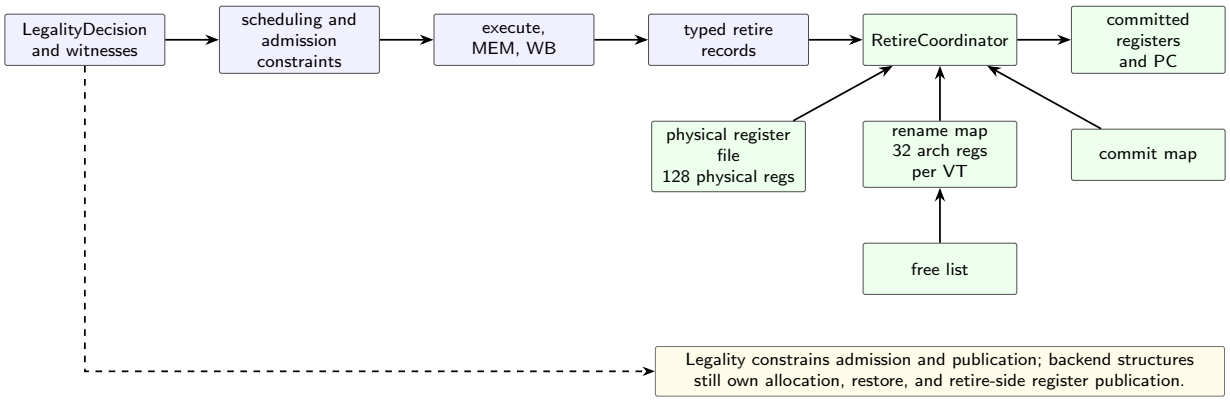

Figure 1: Backend state truthfulness. Typed placement does not replace PRF, rename, commit-map, free-list, or retire-side ownership.

Table 2: Typed-lane topology; branch/control and system-singleton share physical lane 7.

Lane(s)	Slot class	Capacity	Architectural meaning
`0..3`	ALU class	`4`	Class-flexible scalar ALU and selected memory-stream vector compute space
`4..5`	LSU class	`2`	Load/store space
`6`	DMA/stream class	`1`	Singleton DMA/stream carrier
`7`	Branch/control class	`1`	Hard-pinned control class
`7`	System-singleton class	`1`	Shares physical lane 7 with control

with class-facing capacities and the explicit shared physical lane-7 capacity

$$\begin{aligned} \mathrm{Cap}(\mathrm{alu}) = 4, \quad & \mathrm{Cap}(\mathrm{lsu}) = 2, \\ \mathrm{Cap}(\mathrm{dma}) = 1, \quad & \mathrm{Cap}(\mathrm{branch}) = 1, \\ \mathrm{Cap}(\mathrm{system}) = 1, & \\ \mathrm{Cap}_{\mathrm{phys}}(\{\mathrm{branch}, \mathrm{system}\}) = 1. & \end{aligned}$$

Width is therefore a capacity vector over classes rather than a scalar count of vacant positions. The aliased lane-7 pair matters: branch/control work and system-singleton work share one physical location even though they remain distinct classes. The displayed singleton capacities are class-facing; $\mathrm{Cap}_{\mathrm{phys}}(\{\mathrm{branch}, \mathrm{system}\}) = 1$ is the non-additive physical constraint. Hard-pinned placement is architectural for those singleton classes, whereas other classes remain class-flexible and are materialized later within their eligible class masks. Class identity is derived by canonical decode and classification into this typed substrate; it is not published as a raw in-slot `SlotClass` field.

7.1.2 Why the compat bundle is `2048` bits and why that matters

The retained compat transport has a fixed geometric explanation rather than an inferred one. One `VLIW_Instruction` slot image contains four 64-bit words, `word0..word3`, for a total of $4 \times 64 = 256$ bits or 32 bytes. One `VLIW_Bundle` aggregates eight such slot images, so the compat aggregate is 8×32 bytes $= 256$ bytes $= 2048$ bits. This is exactly the transport footprint of the typed `W=8` substrate: one compat slot image per architectural slot position.

The word layout also matters. `word0` is the execution-control word that carries opcode, datatype, predicate, flags, and immediate fields. `word1` is the primary source or address transport. `word2` carries destination and vector-length transport.

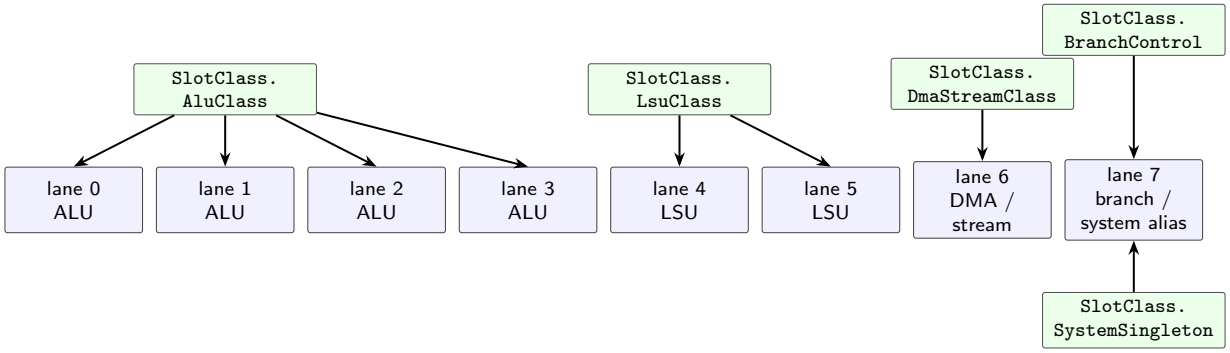

Figure 2: Fixed typed `W=8` lane topology. Width is a class-capacity vector, not eight interchangeable issue positions.

Table 3: Compat transport footprint versus active ingress object.

Entity	Concrete form	Meaning
Slot	`32 bytes / 256 bits`	Compat transport container for one slot image
Bundle	`256 bytes / 2048 bits`	Eight-slot compat aggregate aligned with typed `W=8` substrate
Active ingress object	Canonical decoded bundle	Runtime-facing architectural object and decoded legality carrier

`word3` carries stream, stride, and other transport hints, including the preserved virtual-thread hint. The manuscript should therefore not describe `word1..word3` as DMA metadata only: they are broader transport fields that let one slot co-describe execution and movement-related payloads.

Figure 3 summarizes the bounded vector and stream-carrier reading of those transport surfaces.

Architectural note. The 2048-bit compat bundle is architecturally relevant as a transport-density fact that motivates legality coordination and replay-aware reuse, but it is not by itself a timing, area, or PPA theorem.

That density matters because the working bundle is not a sparse opcode shell. One retained container jointly carries execution-control bits, register-like transport, addressing transport, vector length, stream or stride state, and compat hints across eight positions. Repeatedly rediscovering legality from raw transport images would enlarge the decode-local and runtime coordination surface, which is why the architecture gives first-class status to the canonical decoded bundle, structural identity, and replay-qualified legality reuse. What does not follow is a universal raw `domainTag` field or raw-payload legality authority: domain and legality remain runtime-projected through decoded descriptors, ownership state, and runtime-local structural witnesses.

7.2 4-Way SMT Over a Typed `W=8` Substrate

Above that ingress, the machine performs 4-way SMT over the same typed `W=8` substrate. Four hardware thread contexts share one lane fabric, but SMT does not erase packet structure. The runtime begins from an owner-anchored bundle and may co-pack additional work from sibling threads only when residual class capacity, legality, ownership guards, and later lane materialization all succeed.

Cross-thread coexistence is therefore bundle composition over typed residual capacity. The scheduler does not draw from a generic shared issue pool that forgets packet identity. It reasons over bundle-local structure: class counts, aliased lanes, dependency

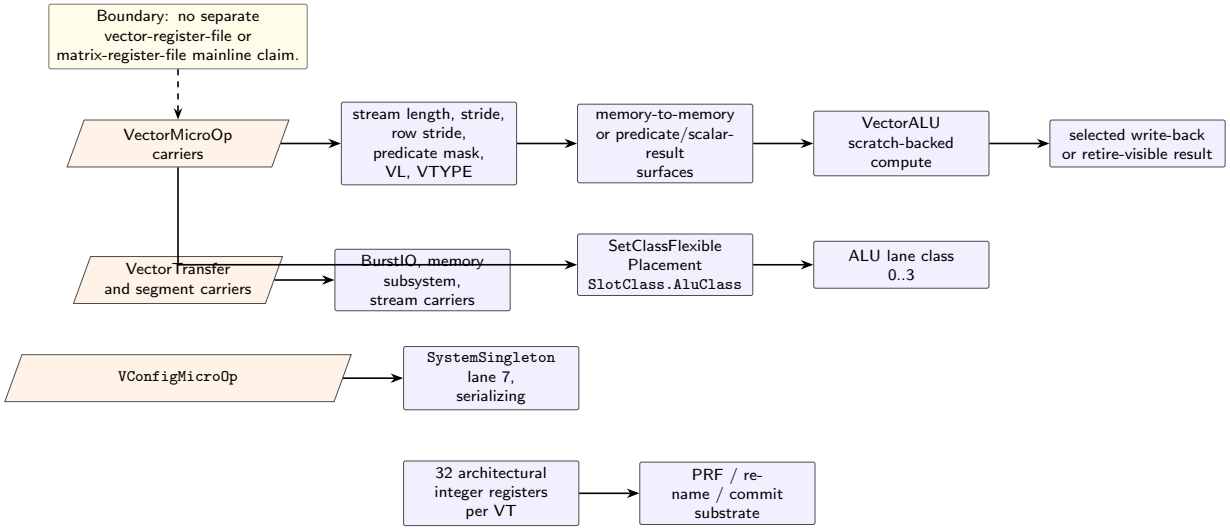

Figure 3: Scalar ALU lanes and selected stream/vector carriers. Vector surfaces are carrier and policy contours over the existing typed substrate.

summaries, ownership scope, and replay-sensitive context. Packet-centric SMT, rather than fetch interleaving alone, is the architectural point.

7.3 Execution Bundle as a Decoded Legality Carrier

The runtime bundle object is a decoded legality carrier. Its safe formal summary is

$$\mathcal{D} = \langle I, O, T, F, V, L_{\text{triage}}, \Delta \rangle,$$

where I is bundle identity, O is occupied-slot state, T is typed-slot structure, F is bundle-level decode state, V is the slot-descriptor family, L_{triage} is per-slot legality triage rather than a final verdict, and Δ is the bundle-local dependency summary.

At the slot level, this carrier preserves more than occupancy. Each occupied slot includes canonical opcode meaning, virtual-thread provenance, read and write register sets, memory/control classification, and placement metadata. At the bundle level, it preserves typed-slot masks, pinning information, dependency summaries, and a bounded cross-thread span summary. The object presented to runtime legality is therefore a structured decoded bundle rather than a width-only packet.

The compat carrier also has a concrete raw slot layout, and that layout explains why the decoded carrier cannot be reduced to an opcode shell. `word0` is the execution-control word, while `word1`, `word2`, and `word3` carry source or address, destination or vector-length, and stream or stride transport-hint payloads respectively. The runtime-facing legality carrier therefore abstracts over a dense compat transport container that co-locates execution and transport facts rather than over a thin instruction token.

That abstraction remains bounded. Domain and legality authority are projected through decoded placement, ownership state, boundary state, and later runtime-local structural witnesses rather than embedded as universal raw slot tags, so the paper does not treat `domainTag` as a raw compat-slot field. Compat transport is the retained container fact; the decoded legality carrier is the active architectural evidence surface consumed by runtime legality and admission.

Decode-side triage remains bounded. It can distinguish direct structural impossibility from cases that require runtime state, but it does not authorize ex-

ecution by itself. Live occupancy, owner/domain scope, boundary state, legality-witness state, and replay context remain downstream runtime questions.

7.4 Versioned Ingress Contract and Staged Typed-Slot Facts

The compiler/runtime contract is explicit and versioned. It is fail-closed only at the ingress/version-handshake boundary, and it is otherwise staged rather than compiler-sovereign. Producers must declare the current contract version before instruction recording or fact publication proceeds, and missing, duplicate, or mismatched declarations are rejected rather than silently tolerated.

This is a two-layer statement rather than a contradiction. The runtime rejects an undeclared or mismatched ingress contract, but typed-slot fact presence inside a successfully declared contract version remains compatibility-tolerant in the current mainline.

Inside that versioned envelope, typed-slot facts remain staged validation surfaces. Compiler preflight emits structural facts about slot class, pinning, and per-class occupancy, and it may also publish bundle-level admissibility summaries. Under the current mainline policy, missing facts remain compatible with canonical execution. Present facts may be validated against the runtime bundle when that bundle is available and otherwise remain structural handoff metadata plus validation/quarantine evidence. Structural disagreement is recorded as mismatch evidence and may be elevated to stricter verification in dedicated proof surfaces, but the current mainline does not treat such facts as mandatory correctness carriers or as a replacement for runtime legality. Compiler-side admissibility summaries therefore remain descriptive build outputs. They do not override a later runtime rejection under live legality, boundary, replay, or pressure state.

This separation matters because the compiler and runtime answer different questions. Compiler preflight answers whether a bundle is structurally plausible under the published topology. Runtime answers whether the presently formed bundle remains admissible under live legality, ownership, replay, and pressure state.

Figure 4 shows the staged handoff without promoting compiler facts into admission authority.

Ingress contract invariant. If a producer artifact reaches canonical runtime ingress, the producer contract version has been declared exactly once and must match `CompilerContract.Version`. Missing typed-slot facts do not invalidate canonical execution in the current mainline. This invariant is supported by the version sentinel and duplicate-handshake checks in `/Core/Contracts/CompilerContract.cs` and `/Core/Processor.CompilerBridge.cs`, together with the validation-only staging surfaces in `SafetyVerifier.TypedSlot.cs` and the compiler-side facts emitter. It is not a claim that compiler typed-slot facts are mandatory admission authority.

7.5 Execution Ownership, Domain Context, and Authority Boundaries

Execution ownership is not the same thing as transport identity. A transport field may record virtual-thread provenance, but that value is insufficient to authorize co-packing, to define retire ownership, or to express cross-domain legality. Runtime ownership is established only after projection into execution carriers through owner thread, owner context, and domain scope.

This distinction is architectural rather than cosmetic. Guard checks reject owner-context drift and domain mismatch before widened admission proceeds, and the same ownership scope carries forward into replay-sensitive reuse and assist transport. The authority boundary is therefore direct: execution is runtime-owned through owner thread, owner context, domain scope, and, where relevant, inter-core provenance; transport metadata alone does not define legal execution.

The paper treats these fields as guard inputs, not as a complete security model. Owner and domain checks bound admission, replay reuse, and assist transport;

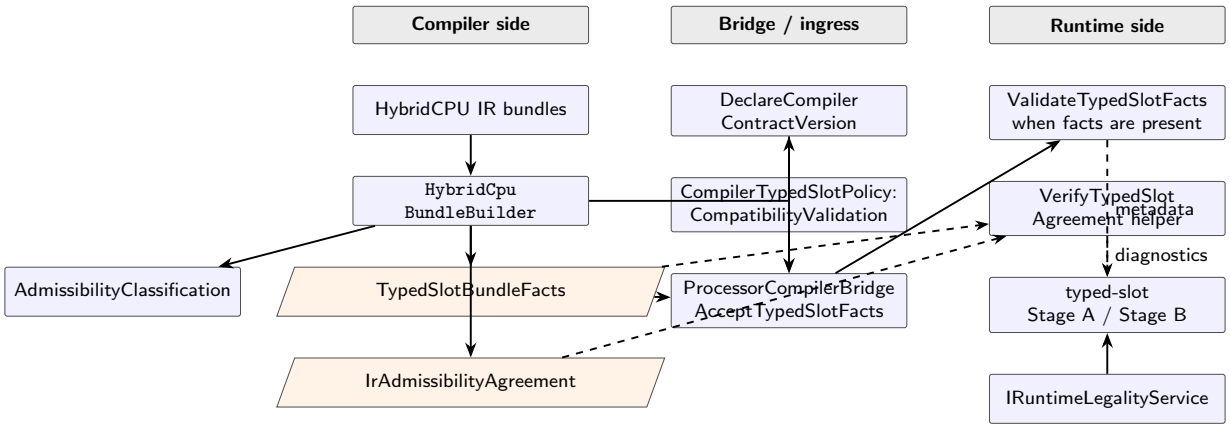

Figure 4: Compiler/runtime fact staging. Compiler facts are validation and quarantine metadata in the current mainline, not live admission authority.

they do not prove repository-wide isolation, covert-channel freedom, or full noninterference.

7.6 Minimal Architectural State and Transition Surface

The preceding sections define the decoded bundle and contract boundary, but those objects are not the whole execution state. The bounded architectural object used in the rest of the paper is the machine state

$$\mathcal{M} = \langle C, P, A, \mu, \mathrm{Mem}, \mathrm{PC}, D, W, Q, B, R, \mathrm{Assist}, \mathrm{Retire}, \mathrm{Fault} \rangle.$$

where C is core-local scheduler, pipeline, and replay-window control state; P is physical register and rename substrate state; A is committed architectural register state; μ is the candidate micro-operation or current admitted carrier; Mem is the bounded architectural memory surface relevant to load, store, atomic, and rollback publication; PC is per-virtual-thread control position; D is the decoded legality carrier; W is the runtime-local structural witness for the current working bundle; Q is typed class capacity and lane occupancy; B is boundary state, including serializing epoch and SMT replay-boundary mode; R is replay phase, template, and token state; Assist is assist nomination, quota, backpressure, and provenance state; Retire is the retire-authoritative publication batch; and Fault is materialized-window fault winner and suppression state.

This tuple is intentionally smaller than a complete ISA semantics. It names only the state required to interpret this paper's claims about legality, admission, replay reuse, assist visibility, retire publication, and fault handling. In particular, D remains a decode-side carrier inside the state, not a substitute for the state itself.

The transition surface is the named one in Table 4. A transition may be an implementation mechanism rather than a fully proved theorem, but each strong claim in later sections is stated against one of these transitions, an implementation-backed invariant, or an explicit non-claim.

The state-boundary convention used below is strict. `Decode` and `PrepareLegality` may classify structure, but they do not authorize execution. `Guard` and the runtime legality service decide whether a candidate is legal under live boundary, owner/domain, replay, and witness state. `AdmitA` and `MaterializeB` are scoped to the typed-slot contour. `InjectAssist` is non-retiring but not generally unobservable. `PublishRetire` is the publication boundary for architectural register, program-counter, deferred-store, and atomic effects. `FaultResolve`

Table 4: Named transition surface for the bounded execution model.

Transition	Primary state boundary	Bounded meaning
`Decode`	D	Forms the decoded legality carrier from canonical ingress; does not authorize execution.
`PrepareLegality`	D, W, Q, B, R	Builds current witness, class occupancy, metadata, and replay-boundary inputs for legality.
`Guard`	B, R, W, μ	Applies boundary and owner/domain checks before replay reuse or witness fallback.
`AdmitA`	Q, W, B, R, μ	Typed-slot class-level admission under residual capacity, ordered legality, and foreground pressure.
`MaterializeB`	Q, μ	Chooses one concrete lane inside the admitted class envelope; does not reopen legality.
`MutateBundle`	D, W, Q, B, R	Inserts an admitted carrier and refreshes or invalidates witness/template state before the next candidate.
`InjectAssist`	Assist, Q, W, B, R	Injects at most bounded auxiliary work under tuple legality, guards, residual capacity, quota, and backpressure.
`Execute`	C, P, μ, Mem	Produces lane-local execute, memory, and write-back carriers; not itself architectural publication.
`PublishRetire`	A, PC, Mem, Retire	Publishes retire records, deferred stores, atomic effects, and serializing-boundary consequences.
`FaultResolve`	Fault, Retire	Selects the materialized-window fault winner and suppresses younger live-subset work.
`ReplayInvalidate`	R, Assist	Clears stale phase, template, lane-hint, witness, or assist nomination state.
`RollbackTokenRestore`	A, Mem, R	Restores only captured architectural registers and exact bound main-memory ranges.

is bounded to the current materialized window and therefore does not constitute a full precise-exception theorem.

Architectural invariant convention. The paper states implementation-backed architectural invariants rather than complete mechanized theorems. Each invariant is operationally tied to a named transition, an implementation surface, and an explicit non-claim. The principal invariants concern ingress contract consistency, guard-before-reuse, witness mutation and refresh, Stage A / Stage B separation, replay-template reuse inside the key, bounded replay-token rollback, assist bounded visibility, bounded retire publication, and materialized-window fault precedence.

8 Execution Bundles, Legality Analysis, and Resource-Legality Witnesses

The architecture overview fixed the execution bundle as the canonical object handed from decode to runtime. This section fixes the legality chain that acts on that object. The architectural separation is between evidence produced from decode-local structure and authority exercised over the live working bundle. Decode can classify occupancy, typed placement, dependency contours, and thread span. Only runtime legality can decide whether coexistence remains lawful under the live owner, domain, boundary, and replay context.

The section uses `resource-legality witness` in a narrow sense. After the boundary-guard chain has

passed, runtime legality may accept or reject through a replay-qualified witness template or through the current structural witness for the working bundle. This witness is a runtime-local carrier of legality-relevant state, not an external attestation artifact and not an independent authorization record for execution correctness.

Figure 5 packages the operational contour used throughout the rest of the paper:

The retained exact-slot compatibility path is described in text rather than collapsed into that contour.

8.1 Decoded Legality Carrier

Legality does not begin from a late scheduler snapshot or from a flat hazard mask. It begins from the canonical decoded bundle and a derived legality carrier

$$\mathcal{D} = \langle I, O, T, F, V, L_{\text{triage}}, \Delta \rangle,$$

where I is bundle identity, O is occupied-slot state, T is typed-slot structure, F is bundle-level decode state, V is the family of occupied-slot descriptors, L_{triage} is per-slot legality triage rather than a final verdict, and Δ is the bundle-local dependency summary.

This carrier preserves exactly the structural facts that later legality needs without importing scheduler-local artifacts. It records occupied slots, typed class and pinning facts, decode flags, cross-thread span, slot-local register footprints, memory/control classification, and bundle-local dependency relations. It is not an execution schedule and not a final verdict. Its role is to make legality inspectable against one stable decoded representation.

In repository terms this decode-side evidence surface is published through `BundleLegalityDescriptor` and its occupied-slot companions. It is distinct from the runtime-local structural witness described below, which records already claimed live resources under the current owner, domain, and boundary state. Likewise, `HasDecodeFault` marks that canonical decoded authority is absent or quarantined; it does not create an alternate legality authority that can override runtime checking.

8.2 Decode-Local Structural Triage Versus Runtime Legality

The first pass over legality is intentionally decode-local. From the slot descriptors, decode derives aggregate read and write masks, aggregate structural-resource state, and pairwise RAW, WAR, WAW, control, coarse memory, system-barrier, and pinned-lane relations. Per-slot triage then partitions peers into two bounded classes: direct structural impossibility and runtime-dependent concern.

The breadth of this triage follows from the slot container itself. `word0` contributes correctness-bearing execution-control bits, while `word1..word3` contribute source or address, destination or vector-length, and stream or stride transport-bearing fields. Combinations such as `Reduction + Is2D + Indexed` therefore enlarge the structural concern surface even before live state is consulted. The safe claim remains narrower than an overload theorem: decode can recognize that such combinations are not a plain scalar case, but current evidence does not license a claim that structural richness alone proves acceptance or rejection. Final resolution still belongs to typed class admission and runtime legality under live pressure, owner or domain guards, and boundary state.

This partition does not usurp runtime authority. Decode-local triage states what follows from bundle structure alone. Runtime legality later interprets the same bundle under live guard, ownership, replay, and witness state. Hybrid CPU therefore rejects both misleading extremes: legality is neither settled entirely at decode nor discovered only after scheduling has already committed to placement.

Ordering-related instruction attributes may influence later legality and retirement handling, but decode-local structure alone does not prove a bundle-wide replay invalidation rule. In particular, the current paper does not claim that one `Acquire` or `Release` attribute in a mixed-SMT bundle automatically invalidates every participant's replay state.

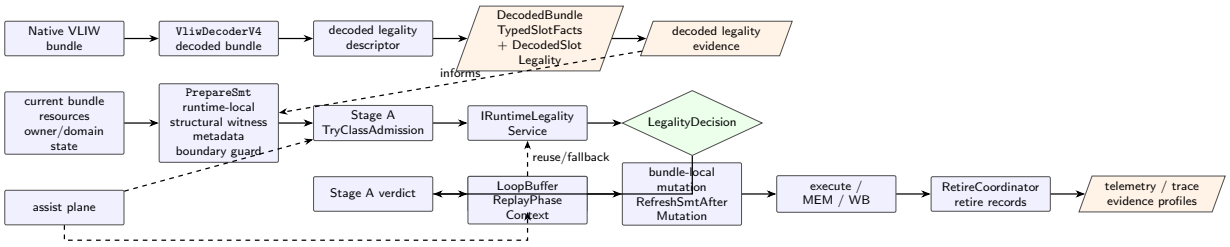

Figure 5: End-to-end contour from native VLIW ingress to telemetry. Decoded descriptors provide evidence; live admission authority remains the runtime `LegalityDecision`.

8.3 Structural Witness Semantics

The live resource model becomes operational through a structural legality witness

$$\mathcal{W} = \langle S, G_0, G_1, G_2, G_3, S_{\text{exec}}, \sigma, n \rangle,$$

where S is shared structural-resource state, G_k is the per-thread register-group state for hardware thread k, S_{exec} is witness-local typed execution occupancy, σ is structural identity, and n is operation count.

This separation is essential. Shared-state conflicts are bundle-wide regardless of virtual thread. Register hazards are checked only within the candidate's own thread partition, and read-after-read is permitted while RAW, WAR, and WAW are not. Witness-local typed execution occupancy is tracked alongside both resource domains so that legality and residual class capacity remain aligned. The witness therefore does not encode one flat scoreboard. It encodes shared structural state, per-thread register-group state, and typed execution occupancy as distinct legality domains.

The witness is a runtime-local legality carrier, not a policy engine. It records what has already been claimed by the current working bundle. It does not rank candidates, choose among placement-feasible lanes, or publish retirement. Operationally, $\sigma(\mathcal{W})$ is also the comparison surface used by replay-qualified reuse: template matching compares structural identity rather than peeking raw masks or ad hoc slot fields one by one. Checker-owned runtime legality remains authoritative over how the witness is interpreted.

Witness mutation invariant. The witness lifetime begins when the working bundle is initialized from the owner bundle. It is mutated only after a candidate has passed runtime legality, typed class admission, and lane materialization for the typed-slot contour. After that mutation, the working bundle, typed occupancy, boundary metadata, and replay legality-cache state are refreshed before another candidate is admitted. The invariant is limited to the modeled typed-slot contour; the exact-slot compatibility path remains a separate compatibility contour unless modeled separately.

8.4 Ordered Guarded Decision Chain

Runtime legality is an ordered guarded decision chain. For the active typed-slot SMT path, the live order is

$$\begin{aligned}
\mathsf{Legal}(\mu, \mathcal{M}) &:= \mathsf{BOK}(\mathcal{M}.B) \wedge \mathsf{ODOK}(\mathcal{M}.B, \mathcal{M}.W, \mu) \\
&\quad \wedge \mathsf{SrcOK}(\mu, \mathcal{M}), \\
\mathsf{SrcOK}(\mu, \mathcal{M}) &:= \mathsf{TplOK}\big(K_{\text{phase}}(\mu, \mathcal{M}), \sigma(\mathcal{M}.W), \\
&\qquad\qquad M_{\text{smt}}(\mathcal{M}), \mathcal{M}.B, \mu\big) \\
&\quad \vee \mathsf{WitOK}(\mathcal{M}.W, \mu).
\end{aligned}$$

Table 5 gives the operational boundary of each predicate. In the formula, BOK, ODOK, and SrcOK abbreviate boundary, owner/domain, and replay-or-witness source acceptance respectively. The dotted arguments are components or derived functions of the current bounded machine state $\mathcal{M}$, not free global symbols. Replay-template reuse is attempted only after the boundary and owner/domain guards

pass; a template mismatch falls back to the current structural witness. Neither witness nor template records independently authorize execution. Implementation names containing `Certificate` denote runtime-local structural witness records; they are not external proof certificates, security attestations, or independent authorization roots.

Figure 6 expands the same order into the runtime legality chain.

This ordering is architectural rather than editorial. Guard rejection is resolved before replay reuse or structural fallback is considered. A candidate that would otherwise satisfy the structural witness remains illegal if the live boundary or owner/domain envelope rejects it. Guard-before-reuse is therefore part of the legality semantics, not an optimization detail.

Guard-before-reuse invariant. For SMT admission, boundary and owner/domain guards are evaluated before replay-template reuse or structural-witness fallback. The implementation anchor is `EvaluateSmtLegality`: boundary guard first, owner/domain guard second, replay-template match third, current structural witness fallback last. This invariant does not prove complete security isolation or hidden-state noninterference; it only fixes the order of the legality decision consumed by admission.

8.5 Structural Identity, Replay Coordination, and Invalidation

Replay-qualified reuse is coordinated through an explicit template key rather than through ad hoc similarity. In repository terms, the reusable key combines `ReplayPhaseKey`, `StructuralIdentity`, `SmtBundleMetadata4Way`, and `BoundaryGuardState`. Replay phase identity answers when reuse is being attempted. Structural identity answers what legality-relevant resource shape is being reused. Bundle metadata and boundary state answer under which owner, domain, and serializing envelope that reuse remains lawful.

Writing K_{phase} for replay phase identity, σ for structural identity, M_{smt} for `SmtBundleMetadata4Way`, and B for boundary state, the guarded reuse key can be summarized as

$$\begin{aligned} K_{\text{reuse}} &:= \langle K_{\text{phase}}, \sigma, M_{\text{smt}}, B \rangle, \\ \mathsf{Match}_{\text{reuse}} &:= [K_{\text{reuse}}^{\text{live}} = K_{\text{reuse}}^{\text{cached}}] \\ &\equiv [K_{\text{phase}}^{\text{live}} = K_{\text{phase}}^{\text{cached}}] \\ &\quad \wedge [\sigma_{\text{live}} = \sigma_{\text{cached}}] \\ &\quad \wedge [M_{\text{smt,live}} = M_{\text{smt,cached}}] \\ &\quad \wedge [B_{\text{live}} = B_{\text{cached}}]. \end{aligned}$$

The predicate is a full-key comparison, not a theorem that partial field agreement is sufficient.

Reuse is consequently conditional and invalidation-driven. If the live template key matches, runtime may reuse replay-qualified witness state. This section intentionally describes legality and class-template reuse rather than `ReplayToken` rollback. If replay phase, structural identity, bundle metadata, or boundary state diverges, reuse is invalidated and legality falls back to the current structural witness or rejects at the boundary-guard chain. Replay-aware legality is therefore invalidation-driven reuse inside a bounded replay envelope, not unconditional memoization and not a token-only decode-elision theorem.

Mutation boundaries are conservative by design. If an immediate field or any other transport-bearing slot field changes in the working bundle, the safe current claim is refresh or invalidation of cached reuse state before legality is reused again. The repository does not publish a fine-grained partial witness-repair theorem for immediate-only mutation, pointer-only mutation, or other subfield edits.

Bounded claim. The manuscript treats immediate or transport-bearing mutation as a reuse-boundary event that triggers refresh or invalidation. It does not claim published partial witness repair.

Ordering surfaces remain part of the guard story, but the current repository does not publish a theorem that one `Release` or `Acquire` in a mixed-SMT bundle invalidates every participant's replay state. That stronger invalidation policy remains open verification work rather than an adopted architectural fact.

Table 5: Ordered runtime legality decision for the typed-slot SMT contour.

Step	Predicate	Operational boundary
G1	BoundaryOK	Rejects closed serializing or SMT boundary modes before candidate-specific structural reuse.
G2	OwnerDomainOK	Rejects owner-context or domain mismatch before any cached replay template is accepted.
R1	ReplayTemplateOK	May reuse a replay-qualified template only when phase, structural identity, SMT metadata, and boundary state match.
S1	StructuralOK	Falls back to the current working-bundle witness when replay reuse is not valid.
Authority	Decision	The legality-service decision is authoritative; neither compiler facts nor the witness alone authorize execution.

Replay-template invariant. Replay-qualified reuse is permitted only if phase key, structural identity, SMT bundle metadata, and boundary state match. Class-template reuse is a separate Stage A capacity shortcut and does not by itself authorize legality. If any member of the replay-template key diverges, the runtime invalidates the cached state or falls back to live legality. This invariant is scoped to legality-template reuse inside the explicit replay key. It does not assert global replay determinism, backend-global rewind, or exact physical lane reproduction.

8.6 Witness Scope and Transition to Admission

The witness in this section has deliberately narrow meaning. The decoded legality carrier and the structural witness together provide a runtime-local legality witness for the current bundle state. They do not imply that work has already been admitted, materialized, executed, or retired.

At this point runtime has produced a legality witness for the current working bundle. Admission policy may then consume that witness to admit one more class claim, materialize one placement-feasible lane, and mutate the working bundle accordingly. The postcondition of a successful `MutateBundle` transition is not merely that one slot contains a new carrier: D, W, Q, B, and the replay-template cache must represent the new working bundle before the next candidate is considered.

Boundary to admission. The witness supplies legality evidence to `AdmitA`. It is not an external attestation artifact, not a compiler artifact, and not a retire authorization record. Its authority is consumed only through the ordered runtime legality service.

9 Two-Stage Admission and Bundle-Compositional SMT Packing

The legality boundary above is consumed here by the policy for the typed-slot path. The mechanism is a bounded typed-slot admission/materialization path over the fixed typed `W=8` substrate, with an explicit exact-slot compatibility path retained as a separate scope boundary. The relevant width is therefore heterogeneous from the outset: `ALU[0..3]`, `LSU[4..5]`, `DMA/stream[6]`, and the aliased `Branch/System[7]` surface.

The packer begins from an owner bundle, derives residual class capacity, and publishes the current legality context of the working bundle before any candidate-specific admission is attempted. A physically empty lane therefore does not imply that the

runtime will use it. Residual capacity remains typed, guard-scoped, and contingent on live legality and pressure state.

All Stage A / Stage B claims in this section are scoped to the typed-slot admission/materialization path. In the current mainline, that path is an opt-in debug/rollout/A-B-comparison contour rather than the only scheduler contour. When it is not selected, the exact-slot compatibility path remains live: it searches a concrete slot first and then consumes legality and dynamic gating over that slot-first opportunity. The Stage A / Stage B terminology does not describe that compatibility contour.

Using the architectural state-transition notation introduced earlier, `AdmitA` is a transition over (Q, W, B, R, μ): it may consume one unit of residual typed class capacity only after the ordered legality decision permits the candidate and the relevant pressure gate accepts it. `MaterializeB` is a later transition over (Q, μ): it binds a concrete lane inside the admitted class mask and currently free lane set. A successful `MutateBundle` then updates D, W, Q, B, and replay-template state. Stage B therefore never serves as a second authority that can widen Stage A legality.

The typed-slot path itself appears in two foreground contours. In the single-cycle contour, nomination, Stage A, Stage B, and bundle-local mutation occur in one pass, followed by a trailing single-cycle intra-core assist pass. In the pipelined foreground contour, the first cycle performs nomination only and the second cycle performs typed-slot admission, lane materialization, and bundle-local mutation. The pipelined contour does not yet integrate the trailing intra-core assist pass.

Figure 7 summarizes the admission/materialization boundary used in this section.

9.1 Admission Consumes Explicit Legality

The typed-slot path separates two questions that should not be collapsed. Stage A asks whether one more unit of typed bundle capacity may be consumed under the current class occupancy, legality witness, and dynamic pressure envelope, without yet choosing a physical lane. Stage B asks which concrete lane, if any, can realize that already admitted class claim. This decomposition is the architectural point of the scheduler contour.

This split is not just pedagogical. One candidate arrives as a dense slot image whose decoded form already reflects execution-control fields, predicate state, immediates, and transport-bearing structure. Re-running a full exact-slot placement analysis for every physically empty lane would repeatedly entangle that dense legality surface with concrete lane occupancy. Stage A therefore asks the narrower and safer question first: whether the working bundle may consume one more unit of the relevant typed class under the current legality context before any lane is chosen.

The pass also contains a bundle-level precondition distinct from candidate admission. Before any sibling candidate is considered, the runtime checks whether the owner bundle is presently open to SMT composition at the current boundary. A negative answer closes the pass before any candidate-specific Stage A or Stage B decision begins.

Bundle-compositional SMT packing is therefore legality consumption followed by typed admission and then lane materialization. After each successful mutation, the working bundle, legality witness, residual class capacity, and projected pressure state are refreshed before the next candidate is attempted. The architectural object remains one evolving bundle rather than a set of anonymous vacancies.

9.2 Stage A: Class-Level Admission

Stage A begins with class capacity rather than lane identity. Residual capacity is a class vector over the fixed typed substrate, not a scalar count of empty positions. If the required class has no remaining capacity, the reject is structural overcommit when the owner bundle already exceeded that class at pass entry, and dynamic exhaustion when earlier successful injections in the same pass consumed the remaining class budget.

Replay-qualified class templates may accelerate this first check, but only in a strictly class-scoped way. A class template records a compatible free-class

Table 6: Architectural transition table for the typed-slot admission/materialization path.

Transition	Input and guard	Result
`PrepareLegality`	owner bundle, residual class occupancy, live legality context	Initializes the working-bundle legality witness and residual class-capacity vector.
`Nominate`	ready non-owner thread contexts	Produces an ordered candidate list; no legality verdict and no lane binding occur here.
`AdmitA.Capacity`	requested slot class, residual class vector, optional replay-qualified class template	Admits or rejects one additional class claim under residual class capacity.
`AdmitA.Legality`	candidate plus current legality witness and guard context	Admits or rejects coexistence under boundary guards, owner/domain guards, replay reuse, or structural fallback.
`AdmitA.Pressure`	scoreboard state, bank-pending state, sampled hardware budgets, speculation budget; non-assist only	Admits or rejects under the current outer-cap pressure envelope.
`MaterializeB`	admitted class claim plus currently free eligible lanes	Materializes one concrete lane inside the admitted class envelope or rejects on pinned-lane or late-binding conflict.
`MutateBundle`	admitted and materialized candidate	Updates the working bundle, legality witness, residual class capacity, boundary metadata, and replay-template state.
`InjectAssist`	single-cycle intra-core contour only	Evaluates at most one assist against residual legality, residual lane capacity, quota, and backpressure after foreground mutation.
`ExactSlotFallback`	typed-slot path disabled	Resolves a concrete slot first and then consumes legality and dynamic gating without class-first admission.

pattern; it does not pre-bind a physical lane and it does not guarantee later placement.

After class capacity, Stage A consumes the runtime legality result. The candidate is evaluated against the current legality witness, owner/context scope, domain scope, boundary state, and replay-qualified legality-witness state. The scheduler may publish a coarse reject family for visibility, but the checker-owned cause remains finer than the scheduler-facing bucket.

For an ordinary foreground candidate μ of class c_μ, the Stage A contour may be summarized compactly as

$$\begin{aligned}\mathsf{Admit}_{A,\mathrm{fg}}(\mu,\mathcal{M}) := (&\mathrm{ResidualCap}(c_\mu,\mathcal{M})>0)\\ &\wedge \mathsf{Legal}(\mu,\mathcal{M})\\ &\wedge \mathsf{OuterCap}_{\mathrm{fg}}(\mu,\mathcal{M}).\end{aligned}$$

Here $\mathrm{ResidualCap}(c_\mu,\mathcal{M}) > 0$ means residual typed-class capacity exists in the current bounded machine state, $\mathsf{Legal}(\mu,\mathcal{M})$ is the ordered guard-plane-first runtime decision exposed through `IRuntimeLegalityService`, and $\mathsf{OuterCap}_{\mathrm{fg}}(\mu,\mathcal{M})$ denotes the foreground dynamic outer-cap envelope checked only after capacity and legality succeed. Assist candidates reuse class-level residual capacity and legality, but they are deliberately not folded into

$\mathsf{Admit}_{A,\mathrm{fg}}$; the assist-specific quota/backpressure section states their governance separately.

Predicate-bearing semantics participate in this same Stage A contour because predicate state is part of the decoded execution-control surface rather than a separate post-admission annotation. The paper therefore does not claim a predicate-only residual-capacity theorem in which a false predicate automatically frees typed capacity or bypasses legality. Predicate-bearing work still enters class admission, legality, and later execution through the same bundle-local envelope.

Only after capacity and legality succeed does the typed-slot foreground path evaluate dynamic outer-cap conditions. These include scoreboard saturation, bank-pending conflicts, sampled hardware memory budgets, and speculation limits for cross-thread memory activity. The result is explicit: a candidate may be structurally legal and still be rejected by the present pressure envelope.

Assist traffic shares the typed substrate but not the exact same Stage A envelope. Assist candidates reuse residual class-level admission and legality, but the ordinary foreground outer-cap gate is skipped for them and replaced by assist-specific backpressure and quota after lane materialization. Mainline foreground admission and assist admission are therefore adjacent, not identical.

Stage A / Stage B separation invariant. Stage A decides only class-level admissibility under residual class capacity, ordered runtime legality, and the applicable pressure surface. Stage B chooses a concrete lane only inside that admitted class envelope. This invariant does not prove port-level timing feasibility or PPA cost, and Stage B does not reopen legality.

9.3 Stage B: Lane Materialization on the Typed `W=8` Substrate

Stage B begins only after Stage A succeeds and therefore does not reopen or widen legality. Its first distinction is between hard-pinned and class-flexible placement. A hard-pinned candidate may materialize only on its declared lane. A class-flexible candidate receives no exact lane commitment from Stage A. Stage B instead intersects the class lane mask with $\mathsf{FreeLanes}(\mathcal{M}.Q)$, the currently free lanes induced by machine-state occupancy, and chooses one concrete lane inside that already admitted class envelope.

Equivalently, Stage B chooses

$$\mathsf{lane}(\mu, \mathcal{M}) \in \mathrm{ClassMask}(c_\mu) \cap \mathsf{FreeLanes}(\mathcal{M}.Q) \cap \mathsf{ReplayMask}(\mu, \mathcal{M}),$$

$$\mathsf{ReplayMask}(\mu, \mathcal{M}) := \begin{cases} \mathsf{StableDonorMask}(\mu, \mathcal{M}), \\ \text{if } \mathsf{ReplayEligible}(\mu, \mathcal{M}), \\ \mathrm{ClassMask}(c_\mu), \\ \text{otherwise.} \end{cases}$$

with the hard-pinned case degenerating to the singleton lane declared by decode. If replay remains active and the previously realized lane is still in that eligible set, runtime may reuse it; otherwise it uses the scheduler's lowest-eligible-lane tie-break. The decision is therefore bounded materialization inside an already admitted class envelope, not a second legality pass.

The replay term is a narrowing hint, not an admission prerequisite. $\mathsf{ReplayEligible}(\mu, \mathcal{M})$ is a boolean guard that selects the stable-donor mask; it is not itself a returned lane mask. In the guard's absence, $\mathsf{ReplayMask}(\mu, \mathcal{M})$ is the identity mask for the class-flexible eligible set and does not block materialization. Stage B therefore chooses a concrete lane inside an already admitted class envelope; it does not reopen legality.

This distinction matters because Stage B is not generic port arbitration or a simple wait-for-port policy. The residual freedom is typed but real: ALU work may use only lanes `0..3`, LSU work only lanes `4..5`, DMA/stream work only lane `6`, and branch/system work the aliased lane `7` surface. Materialization must preserve the typed packet structure already admitted by Stage A under the current class mask, hard pinning, lane aliasing, occupancy, and replay-aware lane constraints. If no lane survives those intersections, Stage B reports a late-binding conflict. The reject therefore means that the current bundle has no packet-structure-preserving real-

ization for that candidate inside the admitted class envelope; it is not ordinary anonymous port pressure and it does not reopen legality beyond Stage A.

9.4 Replay-Aware Lane Reuse Within the Envelope

Replay participation enters the admission path in two bounded forms. At Stage A, a live replay phase may provide a class-capacity template that accelerates the initial class check. At Stage B, replay provides a lane-reuse hint. These mechanisms are related but not interchangeable: one constrains class admission, the other constrains lane choice.

Class-template reuse is valid only inside an explicit replay envelope. Replay deactivation, epoch transition, domain-scope drift, serializing boundaries, or class-capacity mismatch all invalidate the template. The template therefore summarizes a compatible free-class pattern rather than freezing exact slot positions.

Lane reuse is narrower still. Stage B first prunes a class-flexible mask by live occupancy and then, when stable donor structure is available, by replay stable-donor structure. The final rule is local and deterministic: preserve the previous lane for that class when it remains eligible; otherwise choose the lowest eligible lane. This is bounded replay-stable placement, not a theorem of global hidden-state determinism and not restoration of a stored universal scheduler-decision vector. The reused artifact is a bounded lane hint inside a live replay envelope, not a tokenized copy of all prior scheduler choices.

9.5 Packing Policy and Dynamic Pressure

Before candidate-specific admission begins, the scheduler orders nominated non-owner thread contexts. This ordering is policy, not legality. Depending on configuration, it may follow fairness credits or a fixed deterministic order, and it may apply bank-pressure tie-breaking to prefer less congested memory traffic. These choices determine which candidate is attempted first; they do not redefine what counts as legal.

The stronger pressure surfaces appear inside Stage A. Projected outstanding memory state, bank-pending state, sampled hardware budgets, and speculation budgets are all part of a bounded dynamic envelope. They are explicit reject surfaces that prevent extension of the current bundle even when class capacity and legality both succeeded.

The practical consequence is incremental, legality-bounded bundle composition. Residual typed capacity may remain unused even when some physical lanes are empty, because no nominated candidate survives the current ordering, legality, pressure, and materialization constraints.

9.6 Reject Taxonomy Across Layers

The reject story is intentionally multi-layered. Compiler preflight, scheduler visibility, and verifier detail do not share one flattened vocabulary.

Figure 8 gives the same taxonomy as a cross-layer flow.

Two consequences follow. First, compiler-side structural admissibility is a continuation state rather than a guarantee that runtime will never reject. Second, scheduler-visible resource conflict is intentionally coarser than the underlying legality-service diagnosis.

9.7 Scope Boundary

The architecture-defining mechanism of this section is the typed-slot admission/materialization path for intra-core SMT packing. The exact-slot compatibility path remains live as bounded compatibility scope, and the pipelined foreground contour remains narrower than the single-cycle contour because it does not yet integrate the trailing intra-core assist pass. In current mainline terms, the typed-slot contour is live but opt-in, while the exact-slot contour remains the compatibility fallback.

The safe novelty claim is correspondingly narrow. The paper studies the execution law produced by typed class admission, later lane materialization, explicit legality consumption, and bundle-local mutation into the working bundle. It does not claim that the mere existence of a two-stage scheduler, or the

Table 7: Reject taxonomy across compiler, scheduler, and verifier layers.

Layer	Representative outcomes	Interpretation
Compiler preflight	structural admissibility, class-capacity overflow, aliased-lane conflict, invalid typed facts	Compile-time structural evidence; runtime may still reject later on live legality or dynamic state.
Runtime class admission	static overcommit, dynamic exhaustion, resource conflict	Class-capacity or legality-service outcomes during Stage A.
Runtime dynamic pressure	scoreboard, bank-pending, sampled hardware budget, speculation budget	Dynamic outer-cap denials after structural legality passed.
Assist overlay	assist quota, assist backpressure	Auxiliary-plane policy denials on the same typed substrate.
Lane materialization	pinned-lane conflict, late binding conflict	Concrete lane could not be realized after Stage A succeeded.
Verifier detail	boundary, owner, domain, shared-resource, per-thread register-group detail	Finer authority detail that should not be flattened into the scheduler-facing layer.

existence of a retained compatibility path, is itself the contribution.

10 Assist-Coupled Data Movement and Donor Semantics

The foreground typed-slot path described above populates residual typed capacity with retire-visible work. The live runtime also implements a narrower auxiliary plane for data movement and locality transport. This plane is related to runahead, helper-thread, speculative-precomputation, and decoupled access/execute traditions [29, 37–39], but it is not a synonym for generic prefetching and not a helper thread outside the scheduler. It is a typed assist plane whose micro-operations carry explicit kind, execution mode, carrier, donor-source identity, owner binding, and replay metadata. Assist nominations therefore enter the same legality, placement, and ownership world as foreground work, but they remain subordinate to retire-visible execution and may be discarded when their preconditions drift.

The ordering on shared typed lanes is asymmetrical and explicit.

1. Existing bundle occupants are fixed input to the assist pass.
2. Foreground SMT packing runs first and mutates the working bundle.
3. The active single-cycle intra-core assist path then evaluates assists only against residual legality and residual lane capacity.
4. Assist bypasses the ordinary non-assist dynamic outer-cap gate, but after lane materialization it still faces assist-specific backpressure and quota.
5. At most one assist may be injected into a bundle.
6. The pipelined foreground contour does not yet integrate this trailing intra-core assist pass.

These rules define assist as a foreground-subordinate residual-capacity path rather than as a symmetric co-scheduler.

10.1 Assist as an Explicit Auxiliary Plane

The assist plane is typed before admission rather than inferred after execution. The landed runtime

distinguishes assist kind, execution mode, physical carrier, donor source, owner binding, and replay epoch. An assist may therefore be rejected at several distinct surfaces: unsupported tuple, boundary state, owner drift, domain mismatch, lane-materialization failure, backpressure, quota exhaustion, or replay invalidation.

Figure 9 shows the assist plane as residual legality-bound work rather than an alternate scheduler.

Assist is auxiliary in visibility, not in rigor. Unlike auxiliary-work mechanisms whose main distinction is latency tolerance or precomputation coverage [37–39], an assist here is code-declared non-retiring, replay-discardable, and architecturally fault-suppressing, but it still carries slot class, placement, owner, domain, and resource metadata. It occupies real typed capacity and consumes real shared resources. The architecture does not introduce a second informal correctness regime for auxiliary work.

An assist candidate is valid only if all of the following surfaces accept it: the kind/execution/carrier/donor tuple is supported; boundary, owner, and domain guards accept it; residual typed capacity and lane materialization succeed; assist backpressure reserves the relevant carrier resources; assist quota reserves issue and line budget; and replay/provenance freshness remains valid when the transport is consumed. Failure at any one surface rejects the assist without making it retire-visible.

10.2 Donor, Target, Carrier, Owner Binding, and Domain Continuity

Assist semantics depend on roles that must not be collapsed. The donor names the context from which locality is derived. The target names the future consumer context the assist is meant to benefit. The carrier names the physical execution plane. Owner binding names the execution context under which the assist is actually admitted. These roles may coincide in the simplest local case, but legality depends on preserving them separately.

This separation also explains why raw transport cannot be the sole authority. The compat slot can carry source or address, destination or vector-length, and stream or stride transport fields that are relevant to movement-oriented work, but those fields do not by themselves settle donor provenance or execution ownership. Those questions close only after donor-source classification, owner binding, domain continuity, and replay compatibility have been reestablished.

This separation becomes most important when transport crosses thread or core boundaries. Local assists may keep donor scope and target scope aligned. Inter-core assists preserve donor-side provenance while simultaneously binding the transported assist to a target-side owner and domain. Before transport is consumed, donor continuity is rechecked. Before execution, target-side owner, domain, replay epoch, and donor legality are rechecked. Assist therefore preserves provenance and execution ownership simultaneously rather than overwriting one with the other. Freshness is therefore governed by donor provenance, owner continuity, domain continuity, and replay compatibility rather than by any token-local freshness predicate.

A compact summary is

$$\begin{aligned}\mathsf{Fresh}_{\mathrm{assist}}(a) := {} & \mathsf{ProvenanceOk}(a) \\ & \wedge \mathsf{OwnerContinuous}(a) \\ & \wedge \mathsf{DomainContinuous}(a) \\ & \wedge \mathsf{ReplayCompatible}(a).\end{aligned}$$

This freshness conjunction is not `ReplayToken`-local and not raw transport-local. It closes only after the cross-surface provenance, owner, and domain checks still hold at consumption time.

10.3 Carrier Planes and Typed Placement

Carrier choice is an architectural placement decision over the typed substrate. LSU-hosted assists consume LSU-class capacity, while DMA/stream assists consume the singleton stream carrier on lane `6`. Carrier choice therefore determines which typed class must be available and which resource surfaces are touched.

This placement story is not bolted onto a scalar instruction shell. The slot container already co-locates

execution-control with source or address, destination or vector-length, and stream or stride transport. HybridCPU therefore co-describes execution and data-movement-relevant payload within one slot container. The safe formulation remains bounded: not every instruction is a DMA descriptor, not every non-`word0` field is assist metadata, and assist legality still closes only through the supported tuple and carrier matrix.

The landed carrier contour is intentionally narrow.

Architectural note. HybridCPU couples execution and transport metadata within one slot container, but assist legality and freshness remain runtime-governed rather than implied by raw payload alone.

Two consequences follow immediately. First, `Vdsa` is lane-6 DMA only. Second, local scalar assists are not generally lane-6 DMA assists. Lane-6 scalar transport is a specific inter-core contour rather than the default local path.

10.4 Four-Tuple Legality

Assist legality is best stated as a four-tuple:

1. `AssistKind`
2. `AssistExecutionMode`
3. `AssistCarrierKind`
4. `AssistDonorSourceKind`

The first three axes fix the carrier tuple. The fourth closes the donor-source semantics. Later owner, domain, replay, and boundary guards do not enlarge this matrix. They operate only on tuples that are already supported.

Equivalently,

$$\mathsf{LegalAssist}(a, \mathcal{M}) := \mathsf{TupleLegal}(a) \wedge \mathsf{GuardLegal}(a, \mathcal{M}).$$

where $\mathsf{TupleLegal}(a)$ means that the assist belongs to one of the supported four-tuples in Table 9 and $\mathsf{GuardLegal}(a, \mathcal{M})$ collects later owner, domain, boundary, and replay checks against the current bounded machine state. Those later guards may reject a supported tuple, but they do not enlarge the tuple space itself.

Every four-tuple not named in Table 9 is illegal. This is why assist should not be described as a generic prefetch metaphor. The runtime excludes unsupported forms by construction.

10.5 Local, Intra-Core, Inter-Core, and Pod-Level Paths

The local assist path begins from a bounded seed set. Same-thread scalar seeds map to LSU-hosted assist contours. Local cross-thread assist creation is much narrower and is presently legal only for vector-directed contours. Scalar store seeds do not become local scalar assists in the same way.

Inter-core assist transport is a separate provenance-bearing surface. A remote donor first publishes transport metadata carrying donor core, donor pod, donor owner, donor domain, donor virtual thread, and donor assist epoch. A receiver may then attempt pod-local or cross-pod transport theft. At that point the transport is still only a candidate. It can be rejected as stale if donor identity, donor ownership, donor epoch, or domain continuity has drifted.

Only after those checks does the receiving side construct a local assist candidate, recheck legality, materialize a lane, reserve assist-specific backpressure, and reserve assist quota. Nomination is therefore not injection, and transport is not execution.

10.6 Assist-Specific Backpressure and Quota

Assist admission is additionally bounded by explicit auxiliary-plane policy. The bundle quota limits injection to at most one assist per bundle. Separate assist memory quotas bound issue credits and prefetch-line credits. Backpressure then samples shared outer-cap headroom, outstanding-memory pressure, and lane-6-specific stream-register pressure.

These policy layers are independent of tuple legality. A tuple may be legal and still be rejected because the current bundle cannot safely borrow shared pressure. This ordering matters because it keeps assist

Table 8: Assist carrier contour.

Assist kind	Execution mode	Carrier	Present role
`DonorPrefetch`	`CachePrefetch`	`LsuHosted`	Default local scalar assist path
`DonorPrefetch`	`StreamRegisterPrefetch`	`Lane6Dma`	Inter-core scalar transport contour
`Ldsa`	`CachePrefetch`	`LsuHosted`	Default local cold-load contour
`Ldsa`	`StreamRegisterPrefetch`	`Lane6Dma`	Inter-core widened scalar contour
`Vdsa`	`StreamRegisterPrefetch`	`Lane6Dma`	Vector-directed assist contour

Table 9: Assist four-tuple legality matrix.

Assist kind	Execution mode	Carrier	Legal donor-source kinds
`DonorPrefetch`	`CachePrefetch`	`LsuHosted`	`SameThreadSeed`
`DonorPrefetch`	`StreamRegisterPrefetch`	`Lane6Dma`	`InterCoreSameVtSeed`, `InterCoreVtDonorSeed`, `InterCoreSameVtHotLoadSeed`, `InterCoreVtDonorHotLoadSeed`, `InterCoreSameVtHotStoreSeed`, `InterCoreVtDonorHotStoreSeed`, `InterCoreSameVtDefaultStoreSeed`, `InterCoreVtDonorDefaultStoreSeed`
`Ldsa`	`CachePrefetch`	`LsuHosted`	`SameThreadSeed`
`Ldsa`	`StreamRegisterPrefetch`	`Lane6Dma`	`InterCoreSameVtSeed`, `InterCoreVtDonorSeed`, `InterCoreSameVtColdStoreSeed`, `InterCoreVtDonorColdStoreSeed`
`Vdsa`	`StreamRegisterPrefetch`	`Lane6Dma`	`SameThreadSeed`, `IntraCoreVtDonorVector`, `InterCoreSameVtVector`, `InterCoreVtDonorVector`, `InterCoreSameVtVectorWriteback`, `InterCoreVtDonorVectorWriteback`

pacing subordinate to foreground execution and legality rather than letting auxiliary traffic bypass the same resource envelope.

Compactly, assist injection governance may be written as

$$\begin{aligned}\mathsf{AdmitAssist}(a,\mathcal{M}) := \mathsf{LegalAssist}(a,\mathcal{M}) \\ \wedge\, (\mathrm{ResidualCap}(c_a,\mathcal{M}) > 0) \\ \wedge\, \mathsf{QuotaOK}_{\mathrm{assist}}(a,\mathcal{M}) \\ \wedge\, \mathsf{BackpressureOK}_{\mathrm{assist}}(a,\mathcal{M}) \\ \wedge\, \mathsf{PrimaryPriorityOK}(a,\mathcal{M}).\end{aligned}$$

This predicate is an admission and pacing predicate for auxiliary work. It does not assert a post-admission retire-isolation theorem or make assist execution universally hidden; those visibility limits remain the bounded evidence surface stated in the visibility and replay-boundaries section.

10.7 Visibility, Invalidation, and Replay Boundaries

Assist is architecturally non-retiring, not unobservable. Relative to helper-style and precomputation lineage [37–39], the distinction here is the explicit visibility boundary rather than a claim of fully hidden auxiliary execution. It does not become a retire event and does not publish retire-visible architectural state through the ordinary retire path. It may, however, warm or occupy assist-owned cache and prefetch carriers, consume LSU or stream resources, and appear in replay and telemetry evidence. The safe visibility claim is therefore bounded invisibility: assist is invisible to architectural retirement but remains microarchitecturally, replay-wise, and telemetry-wise observable.

Replay qualification is equally explicit. Assist nominations are invalidated on replay transitions,

Table 10: Assist visibility classes.

Visibility class	Positive claim	Explicit limit
Retire-visible architectural state	Assist does not publish ordinary retire records and `IsRetireVisible` is false.	This does not erase carrier, cache, quota, or telemetry effects.
Carrier and resource effects	Assist may occupy LSU or DMA/stream carriers and reserve assist quota/backpressure resources.	Carrier consumption is governed work, not free background execution.
Microarchitectural cache/prefetch effects	Donor-prefetch, LDSA, and VDSA assists may touch prefetch/cache or stream-register surfaces.	The paper does not claim these effects are architecturally invisible in every observation model.
Replay and telemetry effects	Assist is code-declared replay-discardable and explicitly invalidatable; counters and provenance may record it.	The post-admission discard/non-retirement diagnostic is test-local, and `ReplayToken` is not the freshness authority for assist transport metadata.
Fault and exception surface	Assist faults are bounded and suppress retire-visible publication for assist work.	No universal assist exception theorem is claimed.

serializing boundaries, owner drift, and inter-core transport drift. Assist work never becomes architecturally binding in the same sense as retire-visible execution, but it is still governed work: typed, quota-limited, provenance-bearing, and explicitly invalidatable. Replay invalidation can therefore close assist nomination validity, but `ReplayToken` is not the freshness authority for assist transport metadata. Freshness remains governed by donor provenance, owner continuity, domain continuity, and replay compatibility at the point where transport is consumed.

Assist bounded-visibility invariant. Assist micro-ops are not retire-visible and are replay-discardable, but they may consume typed capacity, quota, backpressure, cache or prefetch carriers, stream-register resources, and telemetry slots. The non-claim is equally important: assist is not universally invisible; it is invisible to architectural retirement only.

Diagnostic boundary. The evaluation evidence separates two surfaces. The assistant decision matrix directly validates admission and rejection policy. A separate accepted-then-invalidated lifecycle diagnostic records zero assist retire records, architectural writes, and committed stores, but that diagnostic is explicitly test-local and is not cited as a complete production-retire proof.

11 Replay-Stable Placement, Replay Tokens, and Execution Boundaries

Section 6 established that assist-coupled work is typed, legality-governed, and architecturally non-retiring. The remaining question is how repeated execution regions may reuse prior structure without overstating that bounded visibility as a claim of global machine exactness. This boundary is adjacent to timing-predictable execution work but narrower in scope: HybridCPU does not claim a whole-machine deterministic timing model [10,11,28]. In the present repository, the answer is an explicit replay envelope built from loop-buffer epochs, phase keys, class-capacity templates, guarded legality-witness reuse, bounded replay tokens, and retire-local publication boundaries.

The central claim of this section is therefore deliberately bounded. HybridCPU does not claim exact

reproduction of all hidden backend, cache, queue, or timing state. It claims that repeated regions may reuse specific legality-bearing and placement-bearing conclusions only while the runtime can re-establish the same replay phase, the same guard and ownership regime, and the same bounded rollback surfaces. When those conditions fail, reuse is invalidated and the machine falls back to live legality and admission.

11.1 Replay as Explicit Epoch-Bound Reuse and Invalidation

Replay in the current implementation is an epoch-scoped reuse discipline centered on a loop buffer, not a generic loop optimization and not an after-the-fact observability tool. The loop buffer holds one decoded bundle and advances through explicit load, active, and drain states. Entering an active epoch publishes replay phase identity; leaving that epoch publishes an explicit invalidation reason rather than letting a stale replay window persist implicitly. Scheduler-side replay reuse is therefore phase-driven and invalidation-driven rather than `ReplayToken`-driven.

The runtime-facing replay context carries a small but sufficient identity set: active/inactive state, epoch identifier, cached program counter, epoch length, completed replay count, valid slot count, stable donor mask, and last invalidation reason. These fields are not telemetry decoration. They are the phase facts consumed by admission and legality reuse when deciding whether prior replay-sensitive conclusions remain valid.

The equality key itself is narrower:

$$K_{\text{phase}} = \langle \text{EpochId}, \text{CachedPc}, \\ \text{StableDonorMask}, \text{ValidSlotCount} \rangle.$$

Other `ReplayPhaseContext` fields such as `EpochLength`, `CompletedReplays`, and `LastInvalidationReason` remain contextual evidence and observability surfaces rather than members of the equality relation that decides phase-key reuse.

Figure 10 makes the separation among replay reuse, class-template reuse, and `ReplayToken` rollback explicit.

Replay reuse and replay invalidation are dual outcomes of the same contract. If phase identity, ownership/boundary state, and structural compatibility still match, reuse may proceed. The owner and domain terms are architectural guards rather than a full enclave or capability system, and are cited here only as isolation-adjacent boundary vocabulary [16, 19, 32, 43, 44]. If any of them does not, the runtime invalidates the reuse state and resumes fresh legality checking over current bundle state.

11.2 Replay Phase Keys, Class Templates, and Bounded Lane Reuse

Replay-stable placement is bounded because HybridCPU is a typed-lane machine. The runtime therefore distinguishes replay phase identity from the richer template surfaces used later. Replay phase identity captures epoch, program-counter recurrence, stable donor structure, and valid slot count. The replay-template reuse key K_{reuse} then combines that phase key with structural identity of the current legality witness, `SmtBundleMetadata4Way`, and `BoundaryGuardState`. The class-template capacity shortcut is separate again: it is a Stage A fast path over compatible free-class capacity patterns, not a field inside `PhaseCertificateTemplateKey4Way`.

For clarity, the replay story has distinct surfaces. Replay phase identity, carried by loop-buffer epoch state and replay phase keys, determines when reuse is even being attempted. Replay-template legality reuse determines which previously established legality conclusions remain reusable under matching structural identity, bundle metadata, and boundary state. The class-template capacity shortcut summarizes free-class capacity for Stage A. Lane-reuse hints are narrower again and apply only when the prior lane remains eligible. The `ReplayToken` rollback carrier is separate: it covers explicit captured state, not scheduler reuse decisions.

This separation matters more, not less, in a dense bundle machine. Because one bundle carries a rich control and transport envelope, replay-aware reuse has architectural value, but the safe claim remains bounded: the paper motivates template reuse and

lane-hint reuse, not a token-only or decode-once theorem.

The class-template layer is important because the repository does not require exact lane-for-lane cloning in order to preserve replay stability. Class-capacity templates capture the per-class free-capacity pattern at template capture time and remain valid only while the current free capacity of every relevant class stays at least as large as the captured pattern. Each successful injection decrements the corresponding class budget. Replay-stable class admission therefore preserves a compatible typed envelope rather than an exact physical image of the prior bundle.

Lane reuse is narrower still. For class-flexible work, the runtime may preserve the previous lane for a class only when replay remains active and that lane is still eligible in the current free-lane mask. Otherwise it falls back to the lowest eligible lane. The resulting claim is local and explicit: replay stabilizes lane choice inside the current legal free-lane set, but it does not imply universal physical replay of all hidden state.

Guard-before-reuse remains in force throughout this path. Owner, domain, and boundary checks must still succeed before replay-qualified legality-witness reuse is accepted. Replay therefore constrains reuse; it does not bypass the boundary-guard chain. Replay-phase keys and legality-witness templates are the scheduler-side fast path; if that envelope breaks, the runtime falls back to live guards and live legality rather than restoring a stored universal scheduler-decision vector.

The exact reuse condition used by the paper is K_{reuse} equality: same replay phase key, same structural identity, same SMT metadata, and same boundary guard state, plus a compatible class template when the Stage A class-template shortcut is used. A previous physical lane may be reused only if it remains inside the current eligible free-lane mask. This separates phase identity, template reuse, and lane-hint reuse from `ReplayToken` rollback.

11.3 Replay Tokens as Bounded Rollback Carriers

Replay-sensitive placement and legality reuse address only one part of repeated execution. Some replay obligations concern pre-execution architectural state. The repository addresses that narrower problem through replay tokens. A replay token is a bounded rollback and reproduction carrier for explicit state dimensions that the implementation knows how to capture and restore exactly. A replay token is therefore not a saved scheduler-decision vector.

On the reproduction side, a token may carry execution-configuration and evidence fields such as random seed, vector configuration, memory-size metadata, trace identity, and timestamp. On the rollback side, it carries only selected architectural-register snapshots, exact bound main-memory byte ranges, owner-thread restore context, and side-effect-gating metadata that determines whether rollback support is required. These surfaces are explicit and finite. A replay token does not carry prior lane choices, stable donor masks, legality-witness identity, rename-map state, commit-map state, free-list state, assist-freshness comparators, or arbitrary speculative pipeline images.

Replay-token scope. Loop-buffer phase identity closes epoch-scoped reuse. Class and legality templates close guarded scheduler reuse. `ReplayToken` closes bounded rollback only for the explicit state it captured.

The restore path is intentionally architectural rather than backend-global. Registers are republished through retire-backed architectural publication surfaces, and memory is restored only for fully bound exact main-memory ranges. Capture requires the same exactness: if a requested main-memory window cannot be bound exactly and kept inside the supported in-range surface, the token does not publish a partial snapshot. Partial capture and partial restore are rejected, the restore path therefore fails closed rather than applying partial repair, and the mechanism does not reinstall a prior rename-map image, free-list image, or arbitrary speculative pipeline state.

The safe claim is therefore narrow but useful: replay tokens support bounded rollback only for the explicit architectural state they captured and can fully restore.

11.4 Invalidation Taxonomy and Replay Envelope Boundaries

Because replay reuse is explicit, leaving the replay envelope must also be explicit. Table 11 records the current invalidation vocabulary.

These reasons do real work. They determine which reuse structures must be cleared, which counters advance, and whether previous-lane hints, class templates, or legality templates remain usable. Invalidation is permission to stop reusing stale replay state and to resume live legality and admission. It is not permission to preserve an old equivalence after the facts that justified it have disappeared. The repository therefore supports refresh or invalidation of cached reuse state, not a published partial witness-repair contract for immediate-only or other transport-bearing mutation.

Assist invalidation remains related but distinct. Assist nomination state has its own invalidation surface, and replay deactivation may close assist assumptions. This interaction matters because replay-stable donor structure cannot survive once replay, owner/domain continuity, or the relevant serializing regime has been lost. It does not make assist retire-visible.

11.5 Materialization, Retirement, and Fault-Ordering Boundaries

The current repository requires a strict separation among bundle admission, stage-latch materialization, and architectural publication. An admitted bundle slot is not yet a materialized execution carrier, and a materialized execution carrier is not yet an architecturally retired effect.

Three boundary classes therefore matter.

1. The scheduler boundary decides whether a candidate is admitted and which lane it receives under the current typed and replay-aware constraints.

2. The materialization boundary decides whether that admitted lane becomes live execute, memory, or write-back state with attached owner, domain, fault, and retire-side carriers.

3. The retirement boundary decides whether typed retire records, deferred stores, and atomic retire effects are published to architectural state.

Figure 11 summarizes the materialized-window fault and retire-publication contour.

Point-of-no-return language must remain local to those publication boundaries. The repository does not expose one universal machine-wide point of no return. It exposes bounded publication seams. Architectural effects become binding only when the relevant retire-side carrier is published.

Ordering surfaces are real but must remain bounded in prose. Acquire/release flags and serializing events can influence later legality and retirement handling, but the paper does not claim a repository-proven theorem that one `Release` in a mixed SMT bundle invalidates every participant's replay state. That stronger claim remains open verification work.

Retire order is likewise explicit and narrower than "everything in write-back commits." A retire-authoritative subset is selected, faulted lanes are removed from ordinary retirement, and deterministic ordering is then imposed over the remaining subset. Bundle slot order is the primary retire key, with lane index used only as a deterministic tie-break. Fault precedence is stage-aware, with write-back older than memory and memory older than execute within the currently materialized window, and only younger live-subset work is suppressed.

The safe precise-exception claim is therefore bounded. The code materially implements deterministic retire eligibility, stage-aware fault precedence, and retire-local memory publication for the current materialized window. It does not prove that every hidden backend contour has been rewound or that every point-of-no-return-adjacent interleaving has been closed into a universal theorem.

Table 11: Replay invalidation vocabulary.

Reason	Meaning
`None`	No replay invalidation has been published.
`Completed`	The replay epoch drained normally.
`PcMismatch`	The requested program counter no longer matches the cached replay program counter.
`Manual`	An explicit manual invalidation closed the replay window.
`CertificateMutation`	Working-bundle mutation made cached legality evidence stale.
`PhaseMismatch`	Cached legality-reuse state no longer matches the live replay phase.
`InactivePhase`	A reuse path encountered a phase that is no longer active or reusable.
`DomainBoundary`	Owner/domain scope no longer matches the stored template scope.
`ClassCapacityMismatch`	Live typed-slot class capacity no longer satisfies the captured class template.
`ClassTemplateExpired`	Epoch change or explicit expiry invalidated the prior class template.
`SerializingEvent`	Trap, fence, VM transition, or other serializing boundary closed the reuse window.

Section 7.7 makes the memory and ordering surface explicit: loads, stores, atomics, fences, serializing boundaries, replay-token memory rollback, and fault precedence are all bounded retire-local mechanisms rather than evidence for a complete memory-model theorem.

11.6 Bounded Claims

Replay-stable behavior is supported only inside the explicit replay envelope defined by active replay phase identity, compatible class templates, matching legality-witness identity, matching owner/domain and boundary metadata, and bounded replay-token surfaces. Within that envelope, the runtime can reuse legality and placement conclusions under matching guards. Outside that envelope, the repository does not claim global determinism in the stronger sense studied by timing-predictable machine lines [10, 11, 28]. It reports divergence through invalidation, mismatch, and replay evidence. The envelope is therefore not a decode-once theorem and not a token-only replay theorem.

Rollback language is equally bounded. The repository supports rollback only for the selected architectural registers and exact main-memory ranges that a replay token has explicitly captured, under its owner-thread and side-effect-gating metadata, and can restore as captured state. It does not claim reinstall of prior naming maps, free-list state, or arbitrary speculative state. Exception language is bounded to the current retire-authoritative subset and stage-aware fault ordering for the materialized window. These are substantive boundaries, but they are not whole-machine exactness theorems or backend-global rewind claims.

The same bounded reading applies to memory and ordering. Store, atomic, fence, and serializing surfaces are included because they affect retire publication and replay/assist invalidation. Their presence is not elevated into a full memory-model proof, official ISA-level acquire/release theorem, or universal precise-exception result.

11.7 Bounded Memory and Ordering Semantics

The implementation exposes memory, ordering, atomic, fence, retire, and fault surfaces, but the paper does not promote them into a complete ISA memory-model proof. The architectural claim is narrower: memory-visible effects are published through bounded retire-side mechanisms, and ordering events close the replay and assist envelopes that the earlier sections rely on.

Loads produce register retire records when they

Table 12: Bounded memory, ordering, retire, and fault semantics.

Surface	Operational definition	Explicit limit
Load publication	Register result is captured as a retire record and published by `RetireCoordinator`.	Does not prove a complete load-ordering theorem.
Store publication	Store data is captured as a scalar store retire-window effect or deferred store lane and applied at retire.	Executed store computation is not by itself architectural commit.
Atomic publication	Atomic memory effect is applied through the retire effect; optional register writeback is retired separately.	Does not prove a complete cross-core atomic memory model.
Fence and serializing event	Serializing retire effects close replay and assist reuse surfaces.	Not every acquire/release interaction in mixed SMT is proven as a replay invalidation theorem.
Replay-token memory rollback	Only exact bound main-memory ranges are captured and restored; partial capture/restore fails closed.	No arbitrary memory image or cache-state rollback.
Fault precedence	Current materialized-window fault winner is stage-aware, with younger live-subset suppression.	No complete precise-exception theorem for all hidden backend state.

have a register destination. Stores are not treated as committed merely because a memory micro-op has executed; scalar store publication is captured as a retire-window effect or deferred store lane and applied at the retire publication boundary. Atomics are likewise split: the memory-side atomic effect is applied through the retire effect, and any returned architectural value is published through the retire coordinator. Serializing boundaries, including fences and full-serial events, invalidate or close replay and assist reuse surfaces rather than silently preserving old templates.

Fault handling remains materialized-window local. The implementation selects a stage-aware fault winner from the current materialized window, removes faulted lanes from ordinary retirement, and suppresses younger live-subset work according to that window. This supports a bounded retire-local discipline, not a full theorem that every hidden backend contour is made precise under every possible interleaving.

Memory/order non-claims. The paper does not claim a full acquire/release theorem for mixed-SMT replay invalidation, a complete cache-coherence model, a universal hidden-state rollback model, or a full precise-exception theorem. It claims only the bounded publication and invalidation discipline above, tied to the current retire-side implementation surfaces.

12 Telemetry, Validation, and Evaluation Methodology

Section 7 bounded replay claims to an explicit reuse and invalidation envelope. The present section defines how the repository is measured and what kinds of empirical statements those measurements support. The governing rule is evidence discipline rather than benchmark breadth. A quantity enters the evaluation only if it is exported by the live runtime, recorded in checked-in evaluation artifacts, or supported by named proof and documentation surfaces. A correctness-sensitive claim enters the evaluation only if the corresponding boundary is exercised by a checked-in proof surface or a checked-in boundary document.

12.1 Evidence Philosophy and Measurement Scope

The methodology follows a strict hierarchy.

1. Live runtime exports determine what is actually measurable.
2. Checked-in proof suites determine which correctness-sensitive statements may rely on those measurements.
3. Checked-in validation-boundary documents determine what may be said about breadth of evidence.
4. Historical notes are background only and never outrank the current evidence chain.

The resulting measurement scope is narrower and more concrete than a conventional benchmark chapter. The paper measures global execution counts, per-thread retire counts, per-class admits and rejects, legality provenance, legality-witness pressure, per-thread register-group conflicts, bank-pending pressure, eligibility masking, replay hit/miss/invalidation behavior, replay checks saved, heterogeneous retired width, and assist nomination/reject/inject surfaces. It does not infer unexported quantities indirectly from throughput. Measured transport width and raw slot layout remain descriptive surfaces only; they do not by themselves justify timing, area, or PPA claims.

Zero-valued counters are not interpreted as proof that a mechanism is absent. A zero may mean only that the documented workloads did not exercise that surface. Null or missing fields are treated as unexported for that artifact rather than as architecturally impossible.

12.2 Telemetry as an Architectural Evidence Surface

The runtime export surface is not ad hoc debugging output. It records the same architecture-facing vocabulary used earlier in the manuscript: typed-slot injections and rejects by class, legality provenance, owner/domain-sensitive rejects, shared-resource and per-thread register-group pressure, bank-pending conflicts, eligibility masks, replay reuse and invalidation, heterogeneous retired width, and assist-specific counters.

Figure 12 shows how runtime, trace, harness, and proof-suite evidence reach the paper claim map.

This telemetry matters because it exposes mechanism outcomes directly. When the evaluation discusses reclaim, it does not rely on throughput alone. It can point to class-flexible injections, class-capacity rejects, register-group conflicts, replay hits and invalidations, and assist-specific reject families as explicit runtime observations. The evaluation therefore remains mechanism-facing even when it reports summary metrics.

Trace evidence complements the counter surface. Replay identity, invalidation reasons, estimated checks saved, donor stability, eligibility masks, assist ownership signatures, and assist-specific injections and invalidations remain visible per event rather than collapsed into one opaque score. Replay evidence is thus interpretable as reuse plus invalidation inside a stated envelope.

12.3 Claim-to-Evidence Mapping

Telemetry says what happened; mechanism and boundary documents constrain what may be claimed from those observations. Table 13 packages that link in manuscript form, while the repository evidence map carries the detailed implementation anchors that are intentionally removed from the submission-facing body.

Table 14 is the implementation-facing companion used to keep the claim envelope from exceeding the code. It is not presented as a formal-verification table; it states which mechanisms are directly code-backed and what each mechanism still does not support.

Table 15 separates transition-specific legality evidence from throughput evidence. The long-run IPC and width matrices support realized execution on the measured typed-slot contour; legality claims are instead tied to guard, admission, materialization, replay, assist, rollback, retire, and fault-resolution surfaces.

Table 16 records the targeted diagnostics added for the replay, safety, and assistant claims. These rows are the publication-facing bridge from console telemetry to manuscript wording: `replay-reuse` is the primary replay-template proof, `safety` is the fail-closed negative-control block, and `assistant` separates admission/rejection policy from the test-local post-admission lifecycle diagnostic.

12.4 Experimental Workload Classes and Protocol

The current evaluation surface now has four empirical layers rather than one. The first is a dated smoke-reference baseline used only as a reproducible minimum sanity artifact. The second is the documented runtime sanity matrix used for short-run mechanism cross-checks. The third is a long-run SPEC-like diagnostic harness implemented in `TestAssemblerConsoleApps`, which repeats architecturally shaped reference slices at controlled iteration budgets. The fourth is a targeted architecture-diagnostic layer for replay, safety, and assistant claims. The long-run layer is stronger than smoke sanity and short-run cross-checking, but it remains SPEC-like rather than official SPEC; the targeted layer carries mechanism-specific evidence rather than throughput evidence.

Long-run scaling is deliberately host-side. Large iteration counts are obtained by repeating a known-good static reference slice from the host rather than by inserting an in-image loop backedge into the emitted program. This matters methodologically because it keeps scaling inside a checked transport and legality contour instead of relying on a previously broken post-bundling control-flow trick. Accordingly, the earlier `IPC = 0.25` collapse from that broken in-image loop/control-flow path is treated as a superseded harness defect rather than as architecture evidence.

The workload classes used by the evaluation are therefore separated by empirical role rather than collapsed into one benchmark label:

The protocol is correspondingly strict. A runtime value is used only if it appears in the documented artifacts. A validation-boundary statement is used only if it is backed by checked-in boundary documentation. Historical totals, stale readme counts, informal status notes, and superseded low-IPC harness paths are not promoted into present claims.

12.5 Metrics and Interpretation Rules

The evaluation uses only metrics that can be stated exactly in terms of the exported evidence surface.

Let I_{ret} be retired instructions, C total cycles, C_{ret} retire-active cycles, L_{scalar} scalar lanes retired, L_{non} non-scalar lanes retired, L_{prep} prepared physical lanes, L_{mat} materialized physical lanes, N_{drop} width-drop events, N_{choice} prepared execution choices, H replay-ready hits, N_{miss} replay-ready misses, S_{chk} exported estimated checks saved, and R_c and A_c the reject and admit counts observed for slot class c on the same measurement contour. Let $W_{\mathrm{phys}} = 8$ denote the fixed physical typed-lane substrate, and let W_{eff} denote the workload-conditioned retire-visible heterogeneous surface inferred only for the measured diagnostic envelope. The ratios below are diagnostic counter ratios and are defined only on contours where the displayed denominators are positive.

Raw cycle IPC.

$$\mathrm{IPC}_{\mathrm{raw}} = \frac{I_{\mathrm{ret}}}{C}, \qquad C > 0.$$

This is the primary end-to-end throughput headline.

Retire-normalized IPC.

$$\mathrm{IPC}_{\mathrm{ret}} = \frac{I_{\mathrm{ret}}}{C_{\mathrm{ret}}}, \qquad C_{\mathrm{ret}} > 0.$$

This is a companion retire-bandwidth metric, not a direct physical-width metric.

Retired physical-lane width.

$$\mathrm{LaneRetire} = \frac{L_{\mathrm{scalar}} + L_{\mathrm{non}}}{C_{\mathrm{ret}}}, \qquad C_{\mathrm{ret}} > 0.$$

This is the direct retired physical-lane width metric.

Effective retire-visible surface. The physical topology is `W=8`, but `W=8` is not used as the useful IPC ceiling for the current NativeVLIW diagnostics. In the packed multi-VT profiles, the exported retire counters show $3300/900 = 3.6667$ scalar lanes per retire cycle and $900/900 = 1.0000$ non-scalar lanes per retire cycle in the short matrix, or 4.6667 total retired physical lanes per retire cycle. For this diagnostic envelope the useful retire-visible surface is therefore interpreted as $W_{\text{eff}} \approx 5$: roughly four scalar/ALU-class lanes plus one non-scalar carrier/control lane. This is a workload-conditioned interpretation, not a global peak-IPC theorem for HybridCPU.

Physical lane realization rate.

$$\text{Rate}_{\text{lane}} = \frac{L_{\text{mat}}}{L_{\text{prep}}}, \qquad L_{\text{prep}} > 0.$$

This is the physical lane realization rate exported by the issue-packet counters.

Width-drop share.

$$\text{Share}_{\text{drop}} = \frac{N_{\text{drop}}}{N_{\text{choice}}}, \qquad N_{\text{choice}} > 0.$$

This is the width-drop share.

Replay reuse hit rate.

$$\text{Rate}_{\text{replay}} = \frac{H}{H + N_{\text{miss}}}, \qquad H + N_{\text{miss}} > 0.$$

This is the replay-template reuse hit rate.

Checks saved.

$$S_{\text{chk}} := N_{\text{checks,saved}}.$$

This quantity is interpreted as the exported estimated total of replay checks saved, not as a derived theorem.

Class reject pressure.

$$\text{RejectPressure}_c = \frac{R_c}{A_c + R_c}, \qquad A_c + R_c > 0.$$

This is reject pressure for slot class c when both admit and reject surfaces are exported on the same contour.

These are metric definitions and diagnostic counter ratios, not theorems. Wide-path successes and partial-width issues remain companion counters for how often the path reaches wide realization versus degrading to narrower realized width. Pipeline stalls, `Share_drop`, and `Rate_lane` are interpreted as harness-integrity and realization-integrity checks rather than as standalone architecture claims. Average bundle utilization and NOP density remain scheduler-side context metrics; they are not substitutes for visible retire. Eligibility, bank, hazard, and cross-domain counters are interpreted as structural causes of reject pressure rather than as direct slowdown theorems.

Six interpretation rules follow.

1. Visible retire, assist activity, and pre-retire legality pressure are never conflated.
2. Zero values and missing values are interpreted differently: zero means "not observed in the measured artifact," whereas missing means "not exported in this artifact."
3. Historical slack-recovery terminology is interpreted only as lineage or local vocabulary, not as a separate architecture theory.
4. Replay hits and checks saved are evidence of bounded replay-envelope reuse, not evidence that `ReplayToken` alone avoids decode or legality work.
5. The physical typed substrate width `W=8` is not conflated with the useful retire-visible IPC ceiling of a specific diagnostic envelope; current packed NativeVLIW utilization may be read against $W_{\text{eff}} \approx 5$, not as a global machine peak.
6. Smoke-reference artifacts, short-run sanity matrices, and long-run SPEC-like matrices are

treated as distinct empirical layers with different evidentiary weight.

12.6 Threats to Validity and Evidence Boundaries

The second boundary concerns benchmark language. The long-run harness is credible SPEC-like evidence because it exercises repeated architecturally shaped workload slices over large dynamic retirement counts, but it is not an official SPEC CPU run and it is not a substitute for full application-suite coverage.

The third boundary concerns scaling method. Long-run results are interpreted only from the current host-side repeated-slice harness. The earlier `IPC = 0.25` observation and related low-IPC paths that depended on broken in-image loop or backedge scaling are treated as superseded harness defects, not as valid architecture measurements.

The fourth boundary concerns evidence inventory. Live evidence counts are useful for describing the scale and maintenance of proof surfaces, but counts of files or declared tests are not coverage measures and not pass totals.

The fifth boundary concerns lightly exercised runtime channels. Hard-pinned placement, boundary-guard rejects, eligibility masking, bank-pending pressure, and assist throughput are all instrumented, but many of them remain mostly quiescent in the published matrices. The paper therefore distinguishes implementation presence from strong workload exercise.

The sixth boundary concerns assist evaluation. The code and proof surfaces support assist tuple legality, carrier choice, and governance by quota and backpressure, while the `assistant` diagnostic directly exercises admission and rejection decisions. Post-admission discard and non-retirement are covered only by a test-local accepted-then-invalidated lifecycle diagnostic with explicit zero-retire/write/store counters. That diagnostic is useful evidence for the intended lifecycle boundary, but it is not a complete production-retire proof and it does not support a mature workload-wide assist-throughput story.

The seventh boundary concerns hardware cost. The repository documents structural foreground contours and a pipelined seam for decomposition, but it does not provide measured timing, area, or PPA evidence sufficient for stronger hardware-cost claims. The manuscript therefore treats detailed hardware-cost analysis as outside the present evidence envelope.

The eighth boundary concerns transport and raw-field interpretation. The repository supports an eight-slot compat transport surface and a concrete `word0` layout, but the paper does not promote those facts into a hardware-cost theorem, a raw-field `domainTag` claim, or a direct raw-field `SlotClass` claim.

The ninth boundary concerns ordering interpretation. Ordering surfaces exist, but the current repository evidence does not prove a bundle-wide `Acquire/Release` replay invalidation theorem for mixed SMT bundles.

The tenth boundary concerns rollback interpretation. The repository supports replay-token restoration only for explicitly captured architectural-register and exact bound main-memory surfaces under owner-thread and side-effect-gating context; it does not support claims about backend-global rewind.

Boundary evidence not yet promoted. Table 20 records the negative-evidence surface explicitly. Where rows are not present in the published evaluation artifacts, the paper treats them as code-backed, instrumented, or boundary-surface mechanisms rather than as workload-exercised evaluation results.

13 Evaluation

Section 8 defined evaluation as an evidence problem rather than as a generic benchmark contest. This section therefore reports only checked-in observed surfaces: the documented short-run runtime sanity matrix, the long-run SPEC-like matrix, the documented replay phase pair, the smoke-reference baseline observed on April 18, 2026, and the proof surfaces that justify interpretation of those artifacts. The section

does not claim a fresh repository-wide pass total, and it does not inflate the smoke-reference subset into a validation-closure result.

13.1 Evaluation Questions

The present evaluation asks six bounded questions.

1. Do the documented short-run and long-run matrices show lawful typed-slot reclaim when residual capacity is present?
2. When reclaim is observed, is realized width preserved to retirement rather than lost between admission and materialization?
3. Are observed rejections attributable to explicit legality and legality-witness surfaces rather than opaque scheduler failure?
4. Does the replay pair show replay-phase and template reuse together with explicit invalidation inside the replay envelope, rather than token-only decode avoidance?
5. Which exported channels are strongly exercised by the documented modes, and which remain mostly quiescent?
6. What does the current smoke-reference baseline show, and what does it still not justify?

13.2 Documented Runtime Baseline

Table 21 restates the current checked-in short-run runtime cross-check surface. The table keeps retired-instruction and cycle counts adjacent because each raw IPC entry is the integer-counter ratio I_{ret}/C, for example `168/40` for `vt` and `188/56` for `alu`. These values remain useful as a sanity and mechanism baseline, but they are not the paper's strongest workload evidence. The stronger throughput story now comes from the long-run SPEC-like matrix.

Three immediate observations follow. First, even this short-run matrix does show workload-conditioned reclaim: the single-thread controls remain at zero, while the packed and latency-hiding modes are positive. Second, the observed legality provenance is explicit rather than opaque: the documented modes report structural-witness provenance rather than an unnamed scheduler outcome, but that label must be read as runtime-observed provenance inside a checker-owned authority chain rather than as witness-only sovereignty. Third, the observed `cross-lane conflict` cases belong to late lane realization after class-level admission and therefore denote failure to preserve a typed packet realization in that bundle state rather than generic wait-for-port pressure. Fourth, positive cases are not uniform. `showcase` reclaims less than `lk` and `bnmcz`, which is consistent with a mixed diagnostic workload rather than a single specialized reclaim kernel.

13.3 Long-Run SPEC-like Matrix

The long-run evidence layer comes from the checked-in `TestAssemblerConsoleApps` SPEC-like harness at `1,000,000` configured iterations. This harness explicitly enables the typed-slot contour in `DiagnosticRuntimeSession` by setting `TypedSlotEnabled = true`; the results therefore support the opt-in typed-slot path, not the retained exact-slot fallback. The harness is SPEC-like rather than official SPEC: the current artifact repeats architecturally shaped reference slices for single-thread integer, single-thread vector-probe, VT-packed scalar, and mixed packed profiles, while measuring replay separately through a stable-versus-rotating phase pair. Large iteration counts are produced by host-side repetition of known-good reference slices, not by inserting loop backedges into the emitted program image. The earlier `IPC = 0.25` collapse from the broken in-image loop/backedge path is therefore excluded from current interpretation and treated as a superseded harness defect.

Two IPC metrics are reported, but they are not interchangeable. Raw cycle IPC is the main throughput headline because it uses total cycles in the denominator. Retire-normalized IPC is the companion retire-bandwidth metric `InstructionsRetired / RetireCycleCount`, not the direct physical-width metric. Direct width evidence remains separate in retired physical lanes per retire-active cycle together with the wide-path and realization counters used in

the realized-width discussion. Using both IPC forms together avoids turning retire bandwidth into the sole throughput claim while still preserving a visible retire-side companion metric.

The denominator for useful diagnostic-envelope utilization is also not the physical `W=8` topology. The tested NativeVLIW packed profiles expose an effective retire-visible heterogeneous surface of approximately five useful lanes: four scalar/ALU-class lanes plus one non-scalar carrier/control lane. Under that bounded interpretation, `4.2000` raw IPC is about `84.0%` of the observed effective surface and `4.6667` retire-normalized IPC is about `93.3%`. This does not claim a global HybridCPU peak IPC of five; it only avoids using the eight-lane physical substrate as the useful-IPC ceiling for this diagnostic workload shape.

The invariants around Table 22 matter as much as the IPC values. Across all four reported long-run profiles the harness reports `Pipeline stalls = 0`, `Width-drop share = 0.0000`, and `Physical lane realization rate = 1.0000`. These values matter because they separate architectural throughput from harness-induced decode or control-flow corruption. The VT-packed profiles sustain `4.2000` raw IPC and `4.6667` retire IPC over `5,000,000` cycles and `21,000,000` retired instructions, while the single-thread controls hold `3.3571` raw IPC and `3.6154` retire IPC over `1,555,555` cycles and `5,222,221` retired instructions. This is stronger than the short-run sanity surface, but it remains bounded SPEC-like evidence rather than official SPEC coverage.

13.4 Bundle Realization and Retired Width

The strongest current positive evidence concerns class-flexible packed execution that survives to retirement. Table 23 summarizes the realized-width surface in the current `1,000,000`-iteration SPEC-like artifact, while Table 22 gives the companion throughput and wide-path counts.

This table supports a precise claim and no more. The packed modes retire more physical work per retire-active cycle than the single-thread controls, and the documented artifacts report full realization with zero observed width-drop share. In the current evidence surface, successful class-level admission is not subsequently eroded by an observed preparation-to-materialization collapse.

The long-run SPEC-like matrix strengthens exactly that claim without widening it. Packed modes scale to `4,500,000` wide-path successes with `0` partial-width issues, and the matrix-wide invariants still report zero observed pipeline stalls, zero width-drop share, and full physical lane realization. The paper therefore has stronger evidence that admitted width survives to realized width and to retirement under long dynamic runs, not just under short cross-checks.

Table 24 records the utilization interpretation used for the packed diagnostic modes. The physical substrate remains eight typed lanes, but the current workload envelope does not expose eight independent useful retire slots. The observed retire mix is approximately four scalar lanes plus one non-scalar carrier/control lane, so the fair diagnostic-envelope denominator is $W_{\text{eff}} \approx 5$. On that denominator, the packed modes reach about `84.0%` raw-cycle utilization and `93.3%` retire-window utilization of the effective heterogeneous retire surface.

The same table also marks the limit of this claim. It is strong evidence for the exercised class-flexible path, not for every lane-binding contour. The documented matrix does not show hard-pinned injections as a positive contributor, so the paper does not generalize these width observations into a blanket statement about all placement cases.

13.5 Reject Taxonomy and Strongly Exercised Pressure Surfaces

The current runtime matrix is most informative when read together with typed reject and injection counters. Table 25 summarizes the most exercised pressure surfaces visible in the checked-in modes.

The dominant observed pressure is therefore clear. The documented matrix is driven primarily by class-flexible admission under per-thread register-group legality-witness pressure. The positive reclaim cases are thus not "free width"; they remain mediated by explicit legality rejection surfaces.

The long-run SPEC-like artifacts preserve the same story at larger scale. In the 1,000,000-iteration artifact, vt and max each report 3,375,000 SMT register-group rejects and 2,500,000 class-flexible injects while sustaining 4.2000 raw IPC. These pressure counters are reported as corrected delta-normalized repeated-slice totals, so they are not mixed with cumulative scheduler snapshot sums. The provided artifact does not include lk or bnmcz rows, so memory-oriented pressure is not promoted into the current long-run claim. The architectural point is therefore visible rather than hidden: legality constrains issue, but it does not collapse the exercised execution envelope.

The matrix also marks weaker surfaces with equal clarity. Hard-pinned injections are zero in every documented mode. Guard-dominated rejects, shared-resource witness rejects, and several boundary-sensitive channels are mostly quiescent in the published runtime matrix. That does not negate those mechanisms, but it does constrain the empirical claim surface. The evaluation therefore does not generalize from this matrix beyond the exercised class-flexible path into hard-pinned or other weakly exercised boundary contours.

13.6 Replay Reuse and Invalidation Inside the Envelope

Replay evidence is strongest when stated as template reuse together with explicit invalidation and live-witness fallback, rather than as a universal determinism theorem. The primary replay diagnostic is therefore the targeted replay-reuse run summarized in Table 26. It separates a stable warm-up case from phase-key, structural-identity, and boundary-state invalidation cases.

Across 16,384 template lookup attempts, the diagnostic records 4,095 template hits and 12,289 misses. The stable case records 4,095 hits after one explicit warmup-misses=1 template-construction miss. Each invalidation case records zero hits and 4,096 misses. The aggregate fallback-to-live-witness=12,289 counter equals the replay-template miss count, while witness-accesses=16,384 remains a separate direct-access counter over all lookup attempts.

The replay phase-pair diagnostic remains useful, but only as a sanity/counter check. In the current long replay phase-pair, each phase runs 1,000,000 configured iterations and performs 3,000,000 readiness lookups; both stable and rotating phases report 3,000,000 ready hits and 0 misses, with zero replay-aware cycle, ready-hit, and checks-saved deltas. The rotating case only records additional phase-mismatch invalidations. That is not the primary proof of replay-template reuse. The supported paper claim is narrower: replay-template reuse succeeds after warm-up under a stable key, and phase-key, structural-identity, or boundary-state drift closes reuse and falls back to live witness evaluation. The evidence does not establish token-local DMA freshness logic, ReplayToken as the sole decode-avoidance substrate, or a universal decode-elision theorem.

13.7 Weakly Exercised Eligibility, Bank, Hazard, and Domain Surfaces

Several exported channels are presently better established as instrumented capability than as strongly exercised workload evidence. The clearest example is eligibility masking. The documented runtime modes report zero masked cycles and zero masked ready candidates, yet the proof surfaces still validate export, summary, and comparison of those signals. The correct conclusion is therefore not that eligibility gating is absent, but that it is not a dominant limiter in the published modes.

The same pattern holds for bank, hazard, and cross-domain channels. These surfaces are measurable and validated as channels, but the current documented matrix is dominated by structural-witness pressure rather than by bank or cross-domain blocks. The paper therefore claims instrumented presence and checked-in support, not strong workload exercise.

13.8 Assist-Coupled Observations

Assist evaluation remains governance-oriented in the present evidence surface. The strongest production-

facing diagnostic is the assistant decision matrix, which validates admission and rejection policy rather than throughput. It records one residual-capacity acceptance and five fail-closed rejection causes: quota, backpressure, owner/domain administrator, invalid replay before admission, and primary-stream priority.

The post-admission lifecycle claim is stated with a narrower evidence label. A test-local lifecycle diagnostic accepts an assist, invalidates the replay phase afterward, and records `assist discarded=1`. The same diagnostic records `assist retire records=0`, `assist architectural writes=0`, `assist committed stores=0`, `assist telemetry events=2`, `assist carrier publications=1`, and `foreground retire records preserved=True`. Because this diagnostic does not exercise the production retire path, it supports the modeled lifecycle boundary and the code-backed assist properties, but it is not cited as a complete production-retire proof.

What the present baseline does not provide is a mature quantitative assist-throughput series comparable to the class-flexible reclaim tables above. The paper therefore limits itself to the supported claims: assist admission and rejection policy is directly tested; assist post-admission discard and non-retirement are supported only by code-backed properties plus the explicit test-local lifecycle diagnostic; and workload-wide assist throughput remains weakly evidenced.

13.9 Validation Boundary and Overall Interpretation

The present evidence base supports six positive statements. First, the repository measures far more than aggregate IPC: it exports typed-slot admission and reject counters, legality provenance, legality-witness pressure, replay reuse and invalidation fields, heterogeneous retired width, and a detailed assist vocabulary. Second, the short-run sanity matrix and the long-run SPEC-like matrix tell a consistent story rather than two different ones. Third, the strongest throughput evidence now comes from the `1,000,000`-iteration SPEC-like matrix, where VT-packed profiles sustain `4.2000` raw IPC with `0` observed pipeline stalls, `0.0000` width-drop share, and `1.0000` physical lane realization rate; interpreted against the observed effective retire-visible surface near $W_{\text{eff}} \approx 5$, that is about `84.0%` raw-cycle utilization. Fourth, legality remains visible at scale rather than disappearing behind throughput aggregates. Fifth, replay evidence is strongest when expressed as stable reuse plus explicit invalidation inside the replay envelope. Sixth, the smoke-reference baseline is real and dated: the checked-in observation on April 18, 2026 records 51 passed, 0 failed, and 0 skipped for the documented subset.

The same evidence base imposes ten explicit non-claims. First, the paper does not claim a fresh repository-wide green total. Second, it does not claim strong current exercise of every exported channel. Third, it does not equate the long-run harness with official SPEC CPU or full application-suite coverage, and it does not treat the earlier `IPC = 0.25` collapse from the broken in-image harness path as an architectural throughput bound. Fourth, it does not derive timing, area, or PPA superiority from the eight-slot compat transport surface or its `2048-bit` width. Fifth, it does not promote physical `W=8` topology into eight independent useful retiring IPC slots or claim a global peak IPC of five. Sixth, it does not promote raw compat-slot metadata into a universal raw-field `domainTag` claim. Seventh, it does not treat `ReplayToken` as the sole replay or decode-avoidance substrate. Eighth, it does not claim a repository-proven rule that one `Release` invalidates every participant's replay state in a mixed SMT bundle. Ninth, it does not promote historical documents over live code, checked-in proof surfaces, or current boundary artifacts. Tenth, because the measured SPEC-like matrix enables `TypedSlotEnabled`, it supports the opt-in typed-slot contour and cannot be cited as performance evidence for the exact-slot compatibility fallback.

Accordingly, the evaluation result is not a universal throughput headline. It is a narrower and more defensible statement: the current repository provides a code-backed telemetry surface, a checked-in proof surface, a dated smoke-reference baseline, a short-run sanity matrix, and a stronger long-run SPEC-like matrix that together show legality-witnessed class-flexible reclaim, stable wide-path realization, and re-

play reuse within an explicit invalidation envelope.

13.10 Related Work

HybridCPU is best positioned by architectural axes rather than by forcing it into a single legacy family. The relevant questions are where legality authority resides, what placement means, how replay reuse is defined, how auxiliary work is integrated, and which discipline governs visible completion. On each axis, the current implementation intersects OoO, SMT, VLIW/EPIC, dataflow/spatial, replay-predictability, and helper-execution traditions without collapsing into any one of them [4, 5, 8, 23, 24, 37].

13.10.1 Authority Location and Legality

ROB-centric out-of-order SMT concentrates final admissibility inside rename state, issue structures, and reorder-mediated recovery [1, 8, 9]. Classical VLIW and EPIC move much more authority to compile-time packet formation [5, 23]. HybridCPU places bundle structure at ingress, but keeps final admissibility at runtime. Decoded bundles and compiler evidence describe candidate structure; the boundary-guard chain, legality service, and current legality witness together form the runtime authority chain, but the witness is not authority in isolation.

13.10.2 Placement Semantics

SIMT-like throughput machines organize placement around homogeneous lanes and collective masks, while explicit-dataflow and spatial families organize execution around operand-driven firing or configured compute fabrics [24, 30, 40, 42]. HybridCPU instead treats placement as a typed bundle-composition problem. One phase decides whether a slot class may absorb additional work; a later phase realizes a concrete lane inside that class envelope. The governing object is therefore the legality-bearing typed bundle rather than a homogeneous lane array or a dataflow firing set.

A particularly close point on the VLIW side is the dynamic fault-tolerant heterogeneous-VLIW mechanism of Psiakis et al. [33]. That work also rejects a purely compiler-fixed interpretation of VLIW placement: it dynamically reschedules work over heterogeneous function units, exploits idle issue positions, and extends the search space from the current bundle to the next bundle. Its purpose and correctness boundary, however, are different from those of HybridCPU. Psiakis et al. use residual issue capacity to execute triplicated original and replicated instructions for fault tolerance; their hardware mechanism is organized around a two-bundle scheduling window, function-unit compatibility, RAW/WAW dependency analysis, replication scheduling, voting scheduling, and result comparison. In contrast, HybridCPU does not claim fault-tolerant voting, redundant execution, or PPA superiority over such dynamic VLIW rescheduling mechanisms. Its placement problem is instead a runtime-owned legality/admission problem: residual typed capacity may be consumed by SMT co-composition or by bounded assist work only after the guard chain, legality service, witness or replay-template source, pressure surface, and later lane materialization all agree. Thus, dynamic reuse of idle heterogeneous VLIW resources is prior art; the distinction here is the elevation of residual capacity into a legality-bearing bundle-composition interface that also governs ownership, assist participation, and replay-bounded reuse.

13.10.3 Replay Reuse and Invalidation

Timing-predictable systems give useful contrast for determinism vocabulary, but their goal is not the same as HybridCPU's phase- and template-scoped reuse discipline [10, 11, 28]. HybridCPU makes replay part of execution policy, but only within a bounded envelope. Phase identity, structural compatibility, boundary state, and owner/domain continuity must all match before reuse is accepted; otherwise the runtime invalidates and falls back to live legality. The relevant comparison is thus conditional reuse with explicit invalidation rather than replay as a pure trace aid or as a claim of global exactness. Scheduler-side replay reuse is therefore phase- and template-governed, while `ReplayToken` remains a distinct bounded rollback surface rather than the sole replay substrate.

13.10.4 Assist Integration

Helper-thread, runahead, speculative precomputation, and decoupled access/execute mechanisms often treat auxiliary work as a side structure beside the main issue path [29, 37–39]. HybridCPU integrates auxiliary work more tightly but also more narrowly. Assist micro-operations have explicit kind, carrier, donor-source, owner/domain metadata, and invalidation rules. They consume typed capacity, pass through the same legality contour as foreground work, and remain non-retire-visible. The architectural significance lies in explicit legality and placement, not in a claim that auxiliary work is free.

13.10.5 Retire Discipline and Visible Completion

Many neighboring designs explain themselves primarily through issue formation, while conventional OoO accounts tie visible completion to reorder-mediated publication and recovery [1, 8, 9]. HybridCPU requires an equally explicit account of visible completion. Packet admission, stage-latch materialization, and architectural publication are distinct surfaces. Assist may execute without becoming retire-visible, and replay reuse may remain valid without implying universal rollback or a full precise-exception theorem. On this axis, HybridCPU is best read as a packet machine with explicit retire-side publication discipline.

Taken together, these axes explain the paper's position. HybridCPU borrows explicit bundles from packet machines, shared execution from SMT, recurrence handling from replay systems, and auxiliary data motion from helper-style mechanisms. Its implemented contribution is the way these elements are joined: runtime legality authority over typed bundles, staged placement, bounded replay reuse with invalidation, assist coupling inside the same legality contour, and retire-side publication boundaries that remain explicit.

13.11 Current Limitations and Non-Goals

The current claim is narrower than the historical ambition of the project, and its boundaries are concrete.

- The live machine still relies on a PRF/rename-backed backend substrate. Typed-slot legality and bundle composition constrain execution around that substrate; they do not remove speculative naming, committed naming, or free-list state.
- Compiler-emitted typed-slot facts remain staged evidence rather than mandatory admission authority. Missing facts remain compatible in the current mainline, while present facts remain structured handoff, validation, and quarantine metadata unless a stricter verification mode is selected.
- The typed-slot admission/materialization path is live but not universal. In the current mainline it is an opt-in debug/rollout/A-B-comparison contour, and disabling it reverts execution to the exact-slot compatibility path rather than halting execution.
- The eight-slot compat transport surface implies `256 bytes = 2048 bits` as a container fact only. The paper does not turn that width into a hardware-cost theorem.
- The physical typed substrate is `W=8`, but the current NativeVLIW diagnostic envelope exposes an effective retire-visible heterogeneous surface closer to $W_{\text{eff}} \approx 5$. The paper does not claim a global peak IPC of five, and it does not normalize useful IPC claims against eight independent retiring slots.
- Replay stability is evidence-bounded. The repository supports replay-stable reuse only while replay phase, class-template compatibility, structural identity, owner/domain scope, and boundary state continue to match. This is not a token-only replay claim.

- Replay-token rollback is bounded to explicitly captured architectural-register state, exact bound main-memory byte ranges, owner-thread restore context, and side-effect gating. It does not reinstall a prior rename-map image, free-list image, or arbitrary speculative pipeline state.

- Domain and ownership metadata are projected through decoded placement and admission surfaces. The paper therefore does not claim a raw compat-slot `domainTag` field as a current architectural fact.

- Ordering surfaces exist, but the repository does not prove a bundle-wide `Acquire/Release` replay invalidation theorem for mixed SMT bundles.

- Memory, ordering, atomic, fence, retire, and fault surfaces are described as bounded retire-local mechanisms. The paper does not claim a complete ISA memory model or a full precise-exception theorem.

- Mutation of legality-bearing structure causes refresh or invalidation of cached reuse state. The repository does not publish a partial witness-repair contract for immediate-only mutation.

- The repository does not yet provide measured timing, area, or PPA evidence for the foreground scheduler contours. Detailed hardware-cost analysis is outside the present evidence envelope, so the manuscript is not a PPA or hardware-cost paper in its current form.

- The long-run evaluation surface is stronger than smoke or short-run sanity matrices, but it remains SPEC-like repeated-slice evidence rather than official SPEC CPU or full application-binary coverage.

- The earlier `IPC = 0.25` collapse belongs to a broken in-image loop-scaling/control-flow path. It is treated as a superseded harness defect rather than as an architectural throughput bound.

- Validation is bounded to the current checked-in evidence surfaces. The smoke-reference artifact observed on April 18, 2026 is useful and reproducible, but it is not a fresh repository-wide pass total.

- Assist evidence is strongest for admission/rejection policy, tuple legality, carrier choice, and quota/backpressure governance. Post-admission discard and non-retirement are covered by code-backed properties plus an explicit test-local lifecycle diagnostic, not by a complete production-retire proof. The pipelined foreground contour still lacks the trailing intra-core assist pass, and workload-wide assist throughput remains weakly evidenced.

These limitations also bound adjacent claims. The repository supports bounded retire ordering and stage-aware fault precedence rather than a complete precise-exception theorem. It supports explicit guard and evidence surfaces rather than a finished security or attestation thesis. It supports canonical bundle ingress and typed-slot bundle composition with a retained compatibility path, not a universal multi-frontend or single-path scheduler claim.

13.12 Conclusion

HybridCPU is best read as a bundle-first execution protocol over a live PRF/rename-backed backend substrate rather than as a renamed member of any single prior family. The implemented core contribution is explicit runtime authority over bundle composition: decoded bundles carry structure, runtime legality decides admissibility, typed-slot admission separates class acceptance from lane realization, assist work is governed by the same typed substrate, and replay-aware reuse is permitted only inside an explicit invalidation envelope. Visible completion remains tied to boundary and retire discipline rather than to issue-time success alone.

The empirical closure is likewise stronger than a toy runtime matrix but still bounded in claims. The current long-run SPEC-like harness shows that the VT-packed profiles sustain `4.2000` raw IPC and

4.6667 retire-normalized IPC at 1,000,000 configured iterations with 0 observed pipeline stalls, 0.0000 width-drop share, and 1.0000 physical lane realization rate, while the single-thread controls sustain 3.3571 raw IPC and 3.6154 retire-normalized IPC. Here retire-normalized IPC is the companion InstructionsRetired / RetireCycleCount metric rather than the direct physical-width metric, which remains separately visible through retired-lane and realization counters. The physical substrate remains W=8, but the measured packed NativeVLIW envelope exposes an effective retire-visible surface near $W_{\text{eff}} \approx 5$; under that bounded interpretation, the packed modes reach about 84.0% raw-cycle utilization and 93.3% retire-window utilization of the useful heterogeneous surface. The targeted diagnostics add the missing architectural proof surface: replay-reuse demonstrates lookup/hit/miss/invalidation/fallback behavior with a single stable warm-up miss; safety demonstrates injected fail-closed guard and replay-template negative controls; and assistant demonstrates admission/rejection policy while labeling post-admission discard/non-retirement as test-local lifecycle evidence. This is exactly why the architecture remains observable as an execution mechanism under typed-lane diagnostic workloads, without turning the result into a throughput, PPA, or benchmark-suite superiority claim. The prior IPC = 0.25 collapse does not survive the corrected host-side repeated-slice harness and is treated as a superseded harness defect, not as a bound on the architecture.

The strongest code-backed claim is therefore specific and bounded. The current repository demonstrates analyzable typed bundle composition on the opt-in typed-slot contour with explicit legality provenance, assistant admission/rejection governance, replay-template reuse and invalidation with live-witness fallback, bounded architectural rollback surfaces, bounded retire-local memory/fault publication, and a live PRF/rename-backed backend substrate. It does not claim that rename machinery vanished, that compiler facts are already mandatory for correctness, that assistant lifecycle evidence is a complete production-retire proof, that replay establishes universal hidden-state determinism or backend-global rewind, that the exact-slot fallback is described by Stage A / Stage B, that timing/area/PPA superiority has already been established, that the long-run harness is official SPEC CPU, or that validation has already closed with a fresh repository-wide total.

References

[1] S. Palacharla, N. P. Jouppi, and J. E. Smith, "Complexity-effective superscalar processors," *ACM SIGARCH Computer Architecture News*, vol. 25, no. 2, pp. 206–218, 1997.

[2] O. Mutlu and L. Subramanian, "Research problems and opportunities in memory systems," *Supercomputing Frontiers and Innovations*, vol. 1, no. 3, pp. 19–55, 2015.

[3] D. W. Wall, "Limits of instruction-level parallelism," *ACM SIGARCH Computer Architecture News*, vol. 19, no. 2, pp. 176–188, 1991.

[4] D. M. Tullsen, S. J. Eggers, and H. M. Levy, "Simultaneous multithreading: Maximizing on-chip parallelism," *ACM SIGARCH Computer Architecture News*, vol. 23, no. 2, pp. 392–403, 1995.

[5] J. A. Fisher, "Very Long Instruction Word architectures and the ELI-512," *ACM SIGARCH Computer Architecture News*, vol. 11, no. 3, pp. 140–150, 1983.

[6] E. Özer, S. Banerjia, and T. M. Conte, "Unified assign and schedule: A new approach to scheduling for clustered register file microarchitectures," *Proceedings of the 31st Annual ACM/IEEE International Symposium on Microarchitecture (MICRO)*, pp. 308–315, 1998.

[7] J. E. Thornton, "Parallel operation in the Control Data 6600," *Proceedings of the October 27-29, 1964, fall joint computer conference, part II*, pp. 33–40, 1964.

[8] R. M. Tomasulo, "An efficient algorithm for exploiting multiple arithmetic units," *IBM Journal of Research and Development*, vol. 11, no. 1, pp. 25–33, 1967.

[9] J. E. Smith and G. S. Sohi, "The microarchitecture of superscalar processors," *Proceedings of the IEEE*, vol. 83, no. 12, pp. 1609–1624, 1995.

[10] S. A. Edwards and E. A. Lee, "The case for the precision timed (PRET) machine," *Proceedings of the 44th annual Design Automation Conference (DAC)*, pp. 264–265, 2007.

[11] M. Zimmer, D. Broman, C. Shaver, and E. A. Lee, "FlexPRET: A processor platform for mixed-criticality systems," *Proceedings of the 19th IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS)*, pp. 101–110, 2014.

[12] U. J. Kapasi *et al.*, "The Imagine stream processor," *Proceedings. 2002 IEEE International Conference on Computer Design: VLSI in Computers and Processors*, pp. 282–288, 2002.

[13] W. J. Dally *et al.*, "Merrimac: Supercomputing with streams," *SC'03: Proceedings of the 2003 ACM/IEEE Conference on Supercomputing*, pp. 35–35, 2003.

[14] C. E. Kozyrakis, S. Perissakis, D. Patterson, T. Anderson, K. Asanović, and N. Cardwell, "Scalable processors in the billion-transistor era: IRAM," *Computer*, vol. 30, no. 9, pp. 75–78, 1997.

[15] G. C. Hunt and J. R. Larus, "Singularity: rethinking the software stack," *ACM SIGOPS Operating Systems Review*, vol. 41, no. 2, pp. 37–49, 2007.

[16] J. Woodruff *et al.*, "The CHERI capability model: Revisiting block-based protection for the 2010s," *Proceedings of the 41st Annual International Symposium on Computer Architecture (ISCA)*, pp. 457–468, 2014.

[17] A. Baumann, M. Peinado, and G. Hunt, "Shielding applications from an untrusted OS with Haven," *ACM Transactions on Computer Systems (TOCS)*, vol. 33, no. 3, pp. 1–26, 2015.

[18] H. Esmaeilzadeh, E. Blem, R. St. Amant, K. Sankaralingam, and D. Burger, "Dark silicon and the end of multicore scaling," *Proceedings of the 38th Annual International Symposium on Computer Architecture (ISCA)*, pp. 365–376, 2011.

[19] M. Lipp *et al.*, "Spectre attacks: Exploiting speculative execution," *40th IEEE Symposium on Security and Privacy (S&P)*, pp. 19–38, 2019.

[20] S. Borkar, "Thousand core chips: A technology perspective," *Proceedings of the 44th annual Design Automation Conference (DAC)*, pp. 746–749, 2007.

[21] P. Kongetira, K. Aingaran, and K. Olukotun, "Niagara: A 32-way multithreaded SPARC processor," *IEEE Micro*, vol. 25, no. 2, pp. 21–29, 2005.

[22] A. Snavely and D. M. Tullsen, "Symbiotic jobscheduling for a simultaneous multithreading processor," *Proceedings of the 9th international conference on Architectural support for programming languages and operating systems (ASPLOS)*, pp. 234–244, 2000.

[23] M. S. Schlansker and B. R. Rau, "EPIC: Explicitly parallel instruction computing," *Computer*, vol. 33, no. 2, pp. 37–45, 2000.

[24] K. Sankaralingam *et al.*, "Exploiting ILP, TLP, and DLP with the polymorphous TRIPS architecture," *Proceedings of the 30th Annual International Symposium on Computer Architecture (ISCA)*, pp. 422–433, 2003.

[25] S. Rixner, W. J. Dally, B. Khailany, P. Mattson, U. Kapasi, and J. D. Owens, "Register organization for media processing," *Proceedings of the 6th International Symposium on High-Performance Computer Architecture (HPCA)*, pp. 375–386, 2000.

[26] K. I. Farkas, P. Chow, N. P. Jouppi, and Z. Vranesic, "The multicluster architecture: Reducing cycle time through partitioning," *Proceedings of the 30th Annual IEEE/ACM Inter-*

national Symposium on Microarchitecture (MICRO), pp. 149–159, 1997.

[27] M. Schoeberl *et al.*, "T-CREST: Time-predictable multi-core architecture for aerospace applications," *Journal of Systems Architecture*, vol. 61, no. 9, pp. 449–471, 2015.

[28] J. Reineke, I. Liu, H. D. Patel, S. Kim, and E. A. Lee, "PRET DRAM controller: Bank privatization for predictability and temporal isolation," *Proceedings of the 7th IEEE/ACM international conference on Hardware/software codesign and system synthesis (CODES+ISSS)*, pp. 99–108, 2011.

[29] J. E. Smith, "Decoupled access/execute computer architectures," *ACM Transactions on Computer Systems (TOCS)*, vol. 2, no. 4, pp. 289–308, 1984.

[30] S. W. Keckler, W. J. Dally, B. Khailany, M. Garland, and D. Glasco, "GPUs and the future of parallel computing," *IEEE Micro*, vol. 31, no. 5, pp. 7–17, 2011.

[31] V. Costan and S. Devadas, "Intel SGX explained," *Cryptology ePrint Archive*, Report 2016/086, 2016.

[32] D. Lee *et al.*, "Keystone: An open framework for architecting trusted execution environments," *Proceedings of the Fifteenth European Conference on Computer Systems (EuroSys)*, pp. 1–16, 2020.

[33] R. Psiakis, A. Kritikakou, and O. Sentieys, "Dynamic Fault-Tolerant VLIW Processor with Heterogeneous Function Units," *Microprocessors and Microsystems*, 2022, doi:10.1016/j.micpro.2022.104455.

[34] D. Boggs *et al.*, "Denver: Nvidia's first 64-bit ARM processor," *IEEE Micro*, vol. 35, no. 2, pp. 46–55, 2015.

[35] J. C. Dehnert *et al.*, "The Transmeta Code Morphing Software: Using Speculation, Recovery, and Adaptive Retranslation to Address Real-Life Challenges," *Proceedings of the International Symposium on Code Generation and Optimization (CGO)*, pp. 15–24, 2003.

[36] S. Rusu *et al.*, "A 32-nm 3.1-billion-transistor 12-wide-issue Itanium processor for mission-critical servers," *IEEE Journal of Solid-State Circuits*, vol. 48, no. 1, pp. 13–22, 2013.

[37] O. Mutlu, J. Stark, C. Wilkerson, and Y. N. Patt, "Runahead execution: An alternative to very large instruction windows for out-of-order processors," *Proceedings of the 9th International Symposium on High-Performance Computer Architecture (HPCA)*, pp. 129–140, 2003.

[38] J. D. Collins, H. Wang, D. M. Tullsen, C. Hughes, Y. F. Lee, D. Lavery, and J. P. Shen, "Dynamic speculative precomputation," *Proceedings of the 34th Annual ACM/IEEE International Symposium on Microarchitecture (MICRO)*, pp. 306–317, 2001.

[39] R. S. Chappell, J. Stark, S. P. Kim, S. K. Reinhardt, and Y. N. Patt, "Simultaneous subordinate microthreading (SSMT)," *Proceedings of the 26th Annual International Symposium on Computer Architecture (ISCA)*, pp. 186–195, 1999.

[40] R. Prabhakar *et al.*, "Plasticine: A Reconfigurable Architecture for Parallel Patterns," *Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA)*, pp. 389–402, 2017.

[41] N. P. Jouppi *et al.*, "In-Datacenter Performance Analysis of a Tensor Processing Unit," *Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA)*, pp. 1–12, 2017.

[42] V. Govindaraju *et al.*, "DySER: Unifying Functionality and Parallelism Specialization for Energy-Efficient Computing," *IEEE Micro*, vol. 32, no. 5, pp. 38–51, 2012.

[43] T. Bourgeat *et al.*, "MI6: Secure Enclaves in a Speculative Out-of-Order Processor," *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pp. 51–64, 2019.

[44] V. Costan, I. Lebedev, and S. Devadas, "Sanctum: Minimal Hardware Extensions for Strong Software Isolation," *25th USENIX Security*, pp. 857–874, 2016.

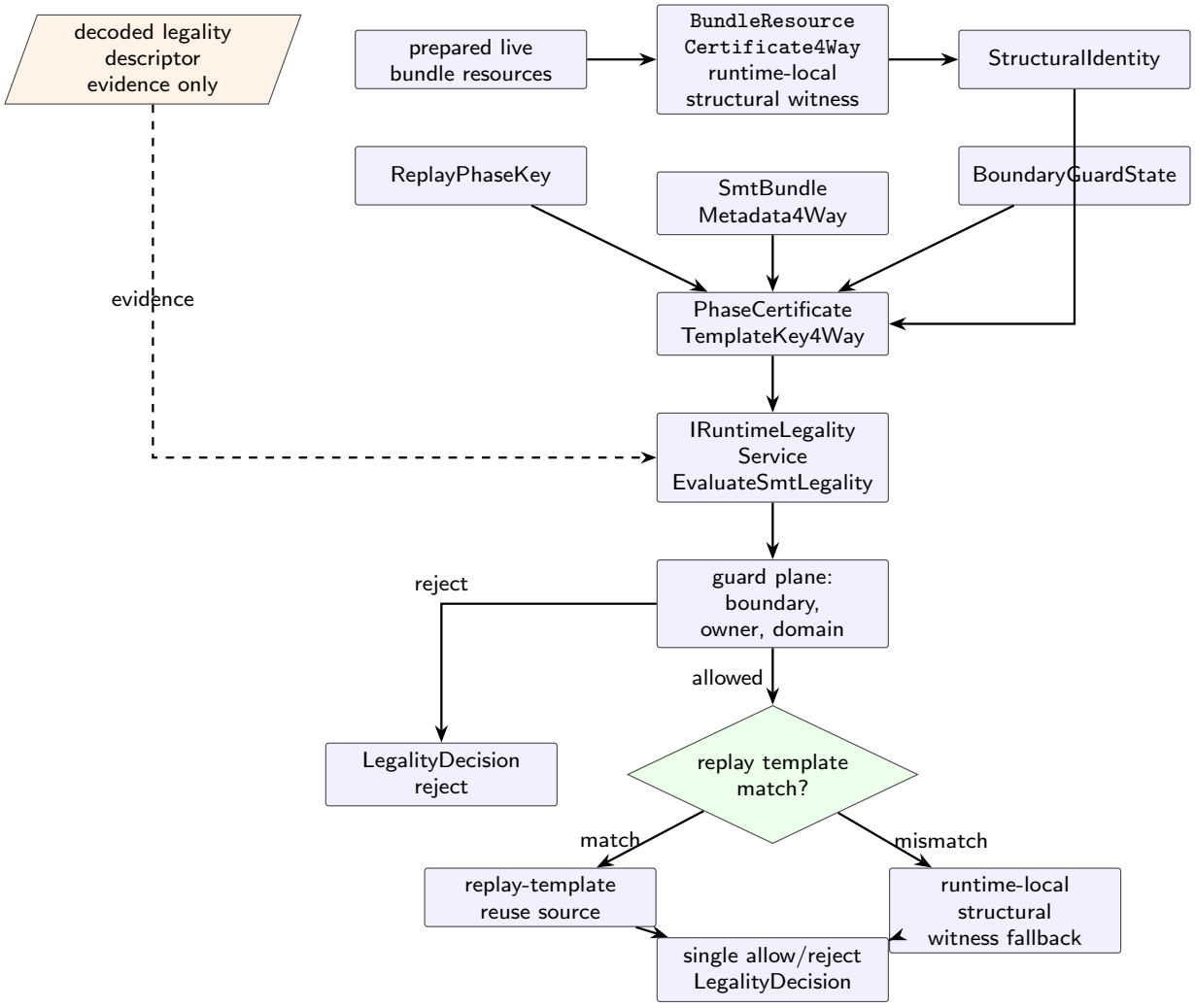

Figure 6: Runtime legality chain. Guard-plane rejection precedes replay reuse, and decoded descriptor facts remain evidence rather than the live witness.

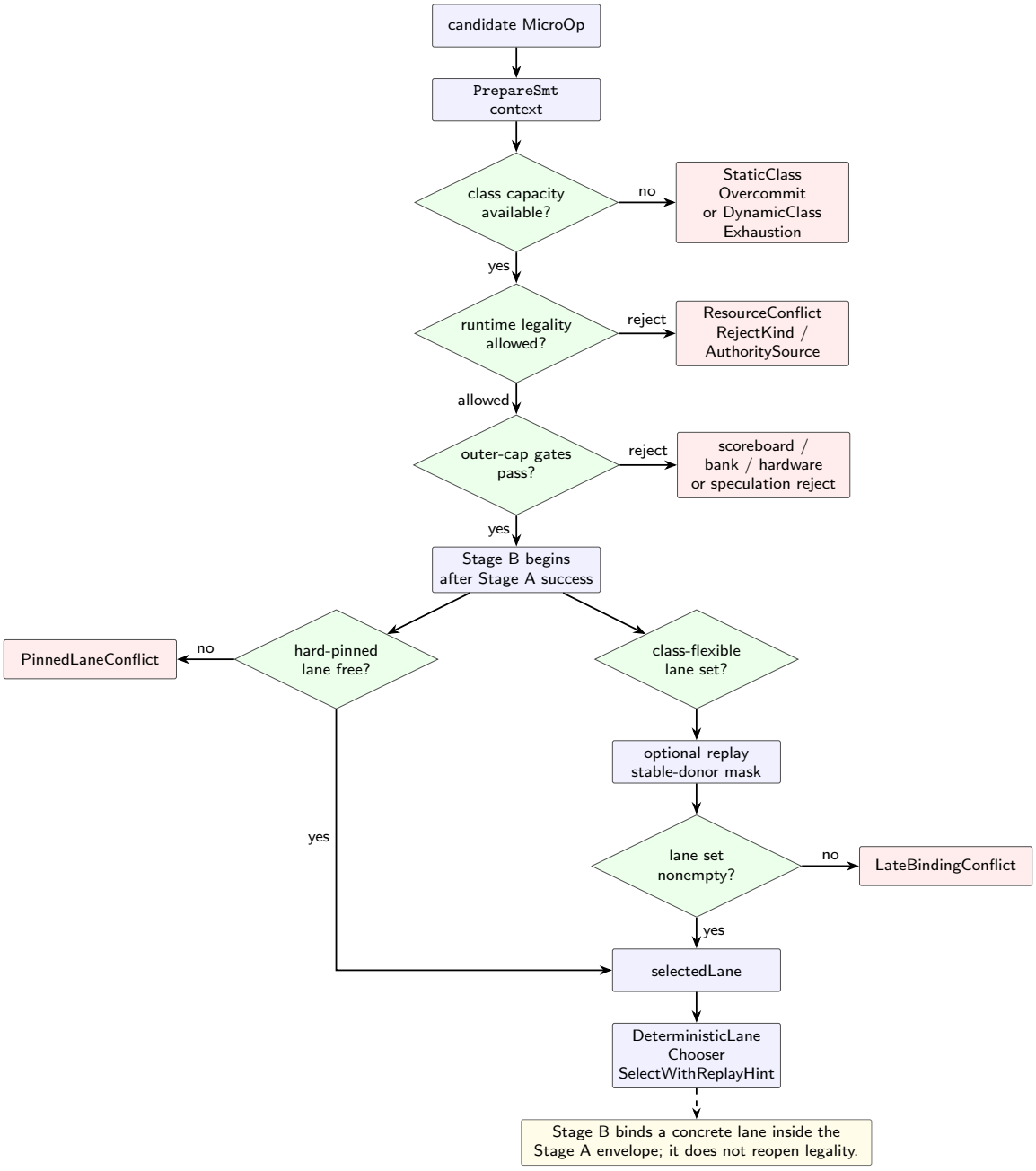

Figure 7: Stage A / Stage B boundary. Stage A admits a class under legality and pressure; Stage B binds a concrete lane without reopening legality.

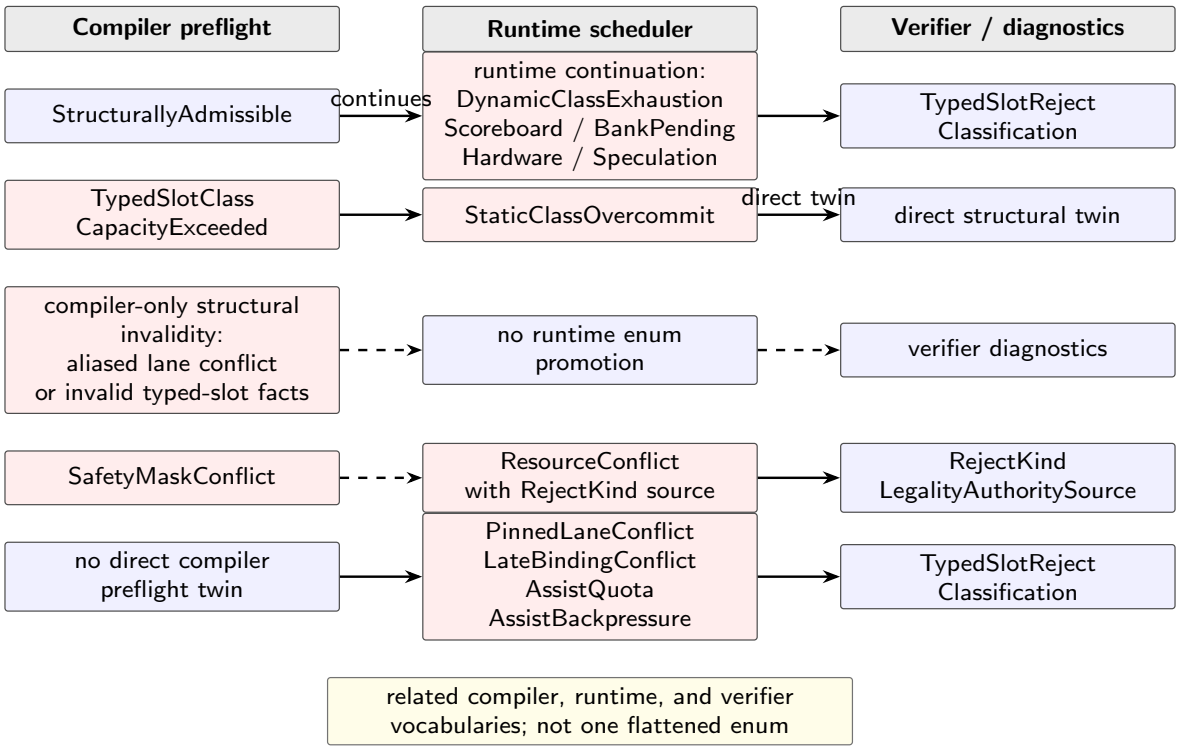

Figure 8: Reject taxonomy across layers. Compiler, runtime, and verifier vocabularies are related surfaces, not one flattened enum.

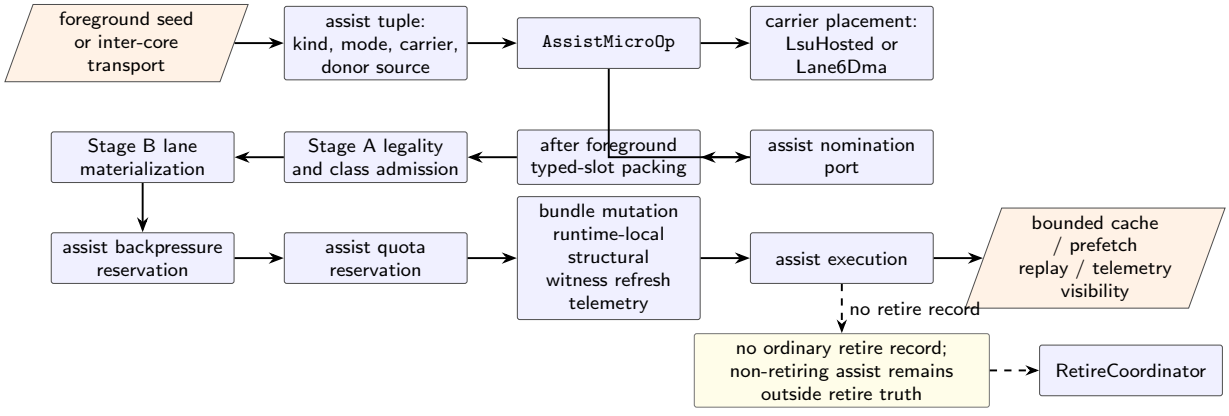

Figure 9: Assist plane as legality-bound residual work. Assist is not a bypass around admission; it is retire-invisible but boundedly observable.

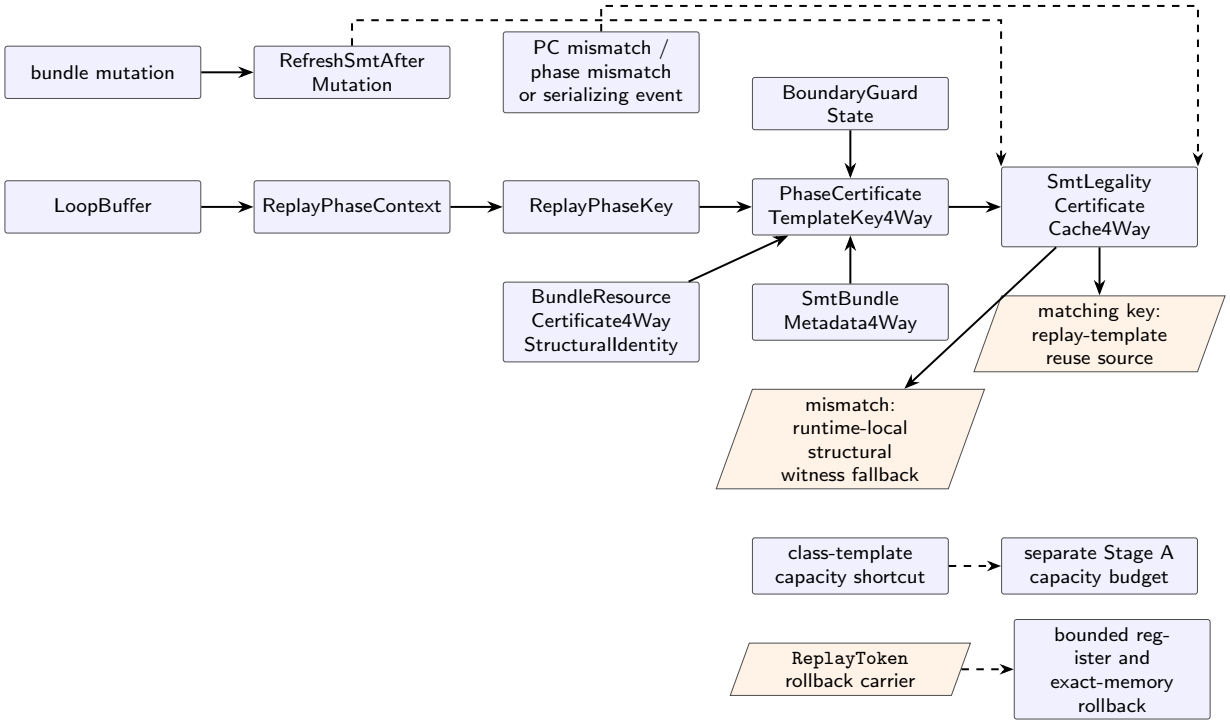

Figure 10: Replay mechanism separation. Replay-template reuse, class-template capacity shortcut, and the `ReplayToken` rollback carrier are distinct bounded surfaces.

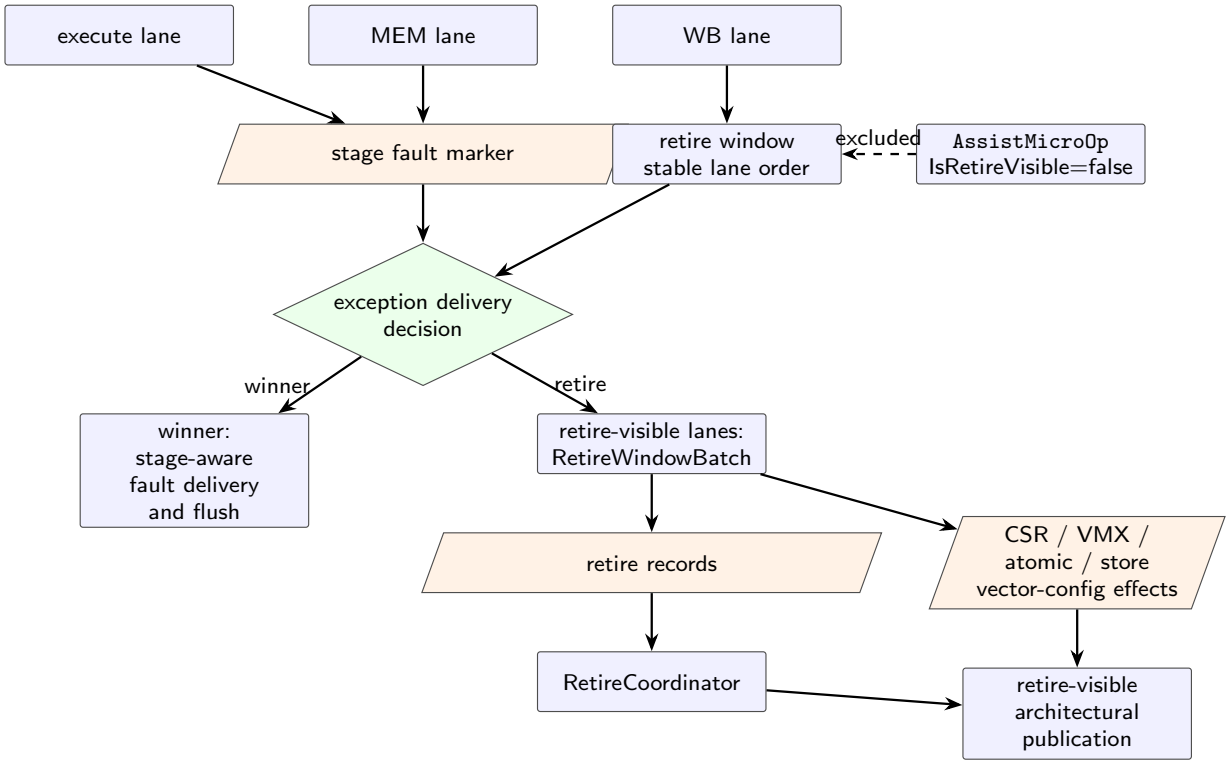

Figure 11: Exception and retire publication boundary. Fault and retire effects are resolved at the write-back and retire boundary.

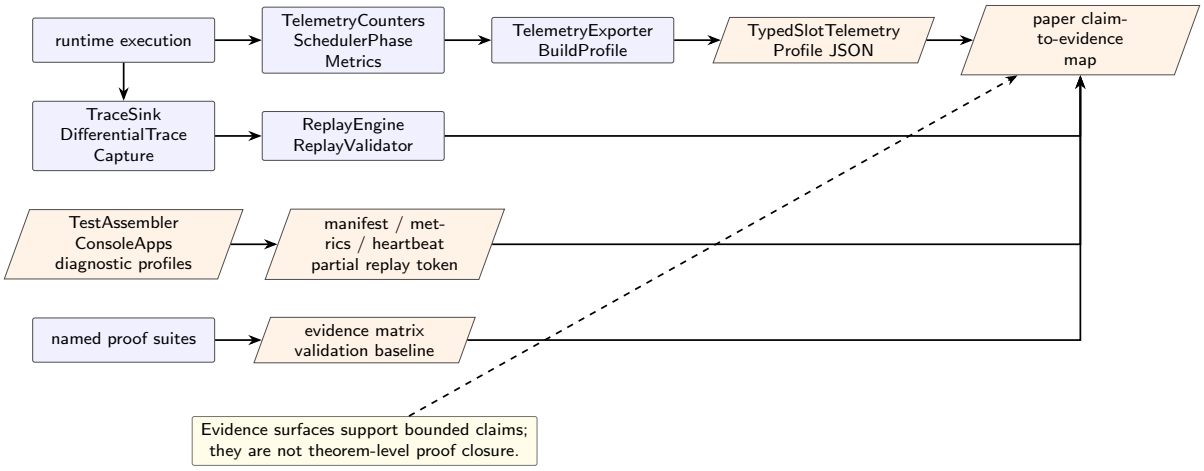

Figure 12: Evaluation and evidence pipeline. Telemetry, traces, diagnostic artifacts, and proof surfaces are evidence channels, not theorem closure by themselves.

Table 13: Claim-to-evidence matrix.

Claim family	Runtime evidence surface	Boundary and documentation surface	Current bounded limit
Fixed typed `W=8` topology, canonical bundle ingress, and staged typed-slot facts	topology-sensitive packing and decode artifacts	topology and ingress boundary surfaces	The ingress/version handshake is fail-closed; `W=8` is physical topology, not a usable eight-lane IPC surface in the current diagnostic envelope, and typed-slot facts remain validation metadata rather than mandatory correctness input.
Compat transport framing and raw slot layout	bundle-size artifacts, decode artifacts, and transport traces	transport-layout and ingress-boundary surfaces	An eight-slot compat bundle implies `256 bytes = 2048 bits` as a container fact, and `word0` layout is real, but the paper does not infer hardware-cost theorems or promote projected domain/class metadata into universal raw slot fields.
Runtime legality and typed-slot admission/materialization	legality provenance, per-class admits/rejects, register-group conflicts, class-capacity rejects, and long-run lane-realization counters	legality-witness and scheduler-boundary surfaces	Runtime legality remains the authority; the Apr. 25 NativeVLIW matrix shows lane materialization preserved prepared lanes in the exercised envelope, not a universal scheduler theorem.
Assist four-tuple legality and visibility boundary	assistant matrix with `6` attempts, one accepted assist, expected quota, backpressure, owner/domain, invalid-replay, and primary-priority rejects, plus test-local lifecycle counters	assist semantics and boundary documents	Admission/rejection is directly tested; post-admission discard and zero retire/write/store counters are test-local lifecycle evidence, and workload-wide assist throughput remains weakly evidenced.
Replay envelope, replay-stable reuse, and bounded rollback	`replay-reuse 16,384` lookup attempts with stable reuse after warm-up, invalidation-case misses/fallbacks, plus phase-pair sanity counters	replay-envelope and rollback-boundary documents	Replay claims remain envelope-bounded and invalidation-driven; `replay-reuse` is the primary template evidence, phase-pair is a counter sanity check, and no token-centric decode-elision theorem is claimed.
Validation breadth and evaluation boundary	smoke-reference artifact, runtime sanity matrix, Apr. 25 long-run SPEC-like matrix, targeted replay/safety/assistant diagnostics, evidence-count inventory	validation-boundary documents and count-maintenance surfaces	The checked-in baseline now includes a stronger long-run SPEC-like counter surface, but it remains bounded and does not become a fresh repository-wide pass total or official SPEC result.

Table 14: Implementation anchor map for bounded architecture claims.

Mechanism	Code/evidence anchor	Claim supported	Claim not supported
Typed `W=8` topology	`SlotClassDefinitions.cs`, `SlotClassCapacity.cs`	Fixed class masks and branch/system aliasing.	Usable eight-lane scalar IPC surface, global peak IPC, or PPA cost.
Decoded legality carrier	`BundleLegalityAnalyzer.cs`	Decode-side occupancy, slot facts, dependencies, and triage.	Full execution semantics by decode alone.
Contract handshake	`CompilerContract.cs`, `Processor.CompilerBridge.cs` `SafetyVerifier.TypedSlot.cs`, `HybridCpuTypedSlot`	Version declaration is fail-closed and duplicate handshake is rejected.	Compiler facts as mandatory admission authority.
Typed-slot facts	`FactsEmitter.cs`	Validation-only facts can be emitted and checked.	Missing facts do not currently reject canonical execution.
Runtime legality chain	`SafetyVerifier.SmtLegality.cs`, `SafetyVerifier.Guards.cs` `BundleResource Certificate4Way.cs`,	Boundary and owner/domain guards precede replay reuse and witness fallback.	Complete isolation or noninterference theorem.
Witness and mutation	scheduler mutation refresh calls	Runtime-local structural witness and refresh after bundle mutation.	External attestation artifact or partial repair theorem.
Stage A / Stage B	`MicroOpScheduler.Admission.cs`, `MicroOpScheduler.SMT.cs`	Class-first admission, later lane materialization, typed-slot opt-in path.	Universal scheduler path; exact-slot fallback remains live.
Assist plane	`AssistMicroOp.cs`, `AssistRuntime.cs`, scheduler assist files, `assistant` diagnostics `ReplayPhaseSubstrate .Implementations.cs`,	Admission/rejection policy, code-backed non-retire properties, and test-local lifecycle discard/non-retirement counters.	Fully invisible assist, end-to-end production retire coverage, or mature throughput theorem.
Replay templates	`replay-reuse` diagnostics	Phase/template/boundary invalidation, guarded reuse after warm-up, and fallback-to-live-witness on misses.	Global replay determinism or token-only decode elision.
Replay token rollback	`ReplayToken.cs` `RetireCoordinator.cs`, `CPU_Core.Pipeline`	Registers and exact bound main-memory ranges restore or fail closed.	Rename maps, free lists, cache state, or arbitrary speculative images.
Retire and memory publication	`Execution.Retire.cs`	Register/PC retire records, deferred stores, atomic retire effects, serializing invalidation.	Complete memory-model or precise-exception theorem.
Evaluation envelope	smoke baseline, short-run matrix, long-run SPEC-like matrix	Mechanism exercise and bounded telemetry observations.	Official SPEC CPU, silicon/PPA superiority, or repository-wide pass total.

Table 15: Transition-to-evidence map for bounded mechanism claims.

Transition	Mechanism claim	Evidence artifact	Not claimed
`Guard`	Boundary and owner/domain checks precede replay-template reuse and witness fallback.	SMT legality verifier, guard surfaces, ordered-legality table.	Complete isolation, security, or noninterference theorem.
`AdmitA`	One typed class claim may be admitted only under residual capacity, runtime legality, and pressure gates.	Typed-slot scheduler path, admission tables, legality provenance counters.	Universal scheduler path or exact-slot fallback performance.
`MaterializeB`	A concrete lane is chosen inside the admitted class envelope; replay narrows only when a stable donor hint is present.	Stage B formula, lane-materialization code, late-binding conflict counters.	Second legality pass or mandatory replay hint.
`MutateBundle`	Successful injection refreshes bundle state, witness state, capacity state, boundary state, and replay-template state.	Scheduler mutation refresh surfaces and structural-witness records.	Partial witness-repair theorem or stale-state authorization.
`InjectAssist`	Assist work is tuple-scoped, guard-checked, quota-limited, and backpressure-limited; post-admission discard/non-retirement is currently test-local evidence.	`AssistRuntime.cs`, assist tuple table, assistant decision matrix, test-local lifecycle counters.	Fully invisible assist, end-to-end production retire coverage, or mature assist-throughput theorem.
`ReplayInvalidate`	Phase, structural, metadata, boundary, domain, or serializing drift invalidates cached replay reuse and falls back to live witness evaluation on template misses.	Replay invalidation vocabulary, replay substrate surfaces, `replay-reuse` diagnostics.	Global replay determinism or token-only decode elision.
`RollbackTokenRestore`	Rollback restores captured architectural registers and exact bound main-memory ranges or fails closed.	`ReplayToken.cs` capture/restore surfaces.	Rename-map, cache-state, free-list, or arbitrary speculative-image restore.
`PublishRetire`	Retire publishes architectural register, PC, deferred-store, atomic, and serializing effects at the visible boundary.	Retire coordinator and retire-side implementation surfaces.	Complete memory model or whole-machine precise-exception theorem.
`FaultResolve`	Fault ordering is bounded to the current materialized window and suppresses younger live-subset work.	Retire/fault ordering surfaces and exception-boundary diagram.	Universal precise exceptions across backend-global state.

Table 16: Targeted replay, safety, and assistant diagnostics.

Diagnostic	Evidence observed	Claim supported	Claim boundary
`replay-reuse`: stable template	`4,096` attempts, `4,095` hits, `1` miss, hit rate `99.98%`.	Stable replay-template reuse after warm-up.	The first miss is template construction, not a failed stable-reuse claim.
`replay-reuse`: invalidation cases	Phase-key, structural-identity, and boundary-state cases each record `0` hits and `4,096` misses; each observes `4,095` invalidation-specific rejects.	Drift in any key component closes reuse.	Does not prove global hidden-state determinism.
`replay-reuse`: fallback accounting	Aggregate `16,384` attempts, `4,095` hits, `12,289` misses; `fallbacks=12,289`; `witness-accesses=16,384`.	Every template miss falls back to live witness evaluation in this diagnostic.	Witness accesses are not the same counter as fallbacks.
`replay`: phase pair	Stable and rotating phases both record `3,000,000` ready hits and `0` misses; each saves `18,000,000` checks; replay-aware cycle delta is `0`.	Sanity/counter check for replay-phase accounting.	Not the primary replay-template proof.
`safety`: negative controls	Owner/context, domain, boundary, invalid-replay-boundary, and stale-witness/template controls all reject; each expected reject counter is observed as `1`.	Fail-closed guard and replay-template rejection surfaces.	These failures are injected; the default SPEC-like matrix does not naturally exercise all guards.
`assistant`: decision matrix	`6` attempts: one accepted residual-capacity assist and one reject each for quota, backpressure, owner/domain administrator, invalid replay, and primary-stream priority.	Assist admission and rejection policy.	Invalid replay here is pre-admission rejection, not post-admission discard.
`assistant`: lifecycle model	Accepted assist is discarded after replay invalidation; retire records, architectural writes, and committed stores are all `0`; telemetry events `2`; carrier publications `1`; foreground retire records preserved.	Test-local post-admission discard and non-retirement visibility counters.	Bounded governance/lifecycle diagnostic evidence, not end-to-end production retire coverage.
Packed NativeVLIW retire surface	Packed modes `vt`, `max`, `lk`, and `bnmcz` record `4.2000` raw IPC and `4.6667` retire-normalized diagnostic width.	High utilization of the observed retire-visible heterogeneous diagnostic surface.	The evidence does not establish a global peak IPC bound.

Table 17: Long-run SPEC-like harness protocol.

Harness property	Current fact
Iteration source	Startup prompt or `--iterations` override
Default matrix suggestion	`250`
Current reported SPEC-like run	`1,000,000` configured iterations
Current artifact	`20260425_021210_724_matrix`, command `matrix`, status `Succeeded`, exit code `0`
Frontend	`NativeVLIW`
Replay suggestion	`1,000,000`
Large-run scaling	Host-side repeated reference slices
Measured workload shapes	`alu`, `novt`, `vt`, `max`, `lk`, and `bnmcz` SPEC-like slices plus targeted `replay`, `replay-reuse`, `safety`, and `assistant` diagnostics
Performance interpretation	Prefer architectural diagnostic counters: cycles, retire cycles, raw IPC, retired width, stall share, lane realization, and wide success rate
Host elapsed time	Report only as run metadata; extended heartbeat telemetry produced multi-GB `heartbeat.ndjson` files, so wall-clock time is not clean model-throughput evidence
Run timestamps	Started `2026-04-25T02:12:10.7992368+00:00`; finished `2026-04-25T16:11:52.2877777+00:00`; elapsed `50381.3072414` seconds
Slice capacities	`36` for single-thread modes; `8` for VT-packed and mixed packed modes in the current artifact
Timeout scaling baseline	`100` iterations
Timeout cap	`512x` the profile base timeout

Table 18: Empirical layers and their evaluation roles.

Workload layer	Representative modes	Primary stress point	Evaluation role
Smoke baseline	dated smoke-reference subset	reproducible minimum sanity artifact	validation floor only
Short-run runtime sanity matrix	`showcase`, `vt`, `max`, `novt`, `alu`, `lk`, `bnmcz`	mechanism-facing cross-check of reclaim, legality, width, and assist channels	short-run sanity and micro cross-check surface
Long-run SPEC-like execution matrix	`alu`, `novt`, `vt`, `max`, `lk`, `bnmcz`	end-to-end throughput over repeated reference slices	main scale and throughput evidence for the current `1,000,000`-iteration artifact; not official SPEC coverage
Targeted architecture diagnostics	`safety`, `replay-reuse`, `assistant`	fail-closed guards, replay-template reuse/fallback, and assist admission policy	primary diagnostic evidence for replay/safety/assist claims within their envelopes
Replay phase pair	`replay` stable versus rotating	replay-phase counter sanity under stable and rotating keys	counter evidence with zero replay-aware cycle delta, not a global replay guarantee

Table 19: Evidence-boundary table.

Evidence surface	Current repository fact	Allowed manuscript use	Boundary statement
Runner-aligned smoke reference	Checked-in smoke artifact observed on April 18, 2026 reports 51 passed, 0 failed, 0 skipped	Claim a reproducible smoke baseline	This is not a repository-wide pass total.
Runtime sanity matrix	Checked-in short-run modes export reclaim, legality, width, and replay observations	Report bounded mechanism-facing behavior for those modes	This matrix is a sanity surface, not the strongest throughput evidence.
Long-run SPEC-like matrix	Artifact `20260425_021210_724_matrix` succeeded under `NativeVLIW` with `1,000,000` configured iterations for `alu`, `novt`, `vt`, `max`, `lk`, and `bnmcz` slices	Report stronger counter-based scale evidence for those workload slices: cycles, retire cycles, IPC, retired width, stall share, lane realization, and wide success rate	This is SPEC-like, not official SPEC, not full application-suite coverage, and not a global peak-IPC bound.
Host wall-clock time	The same run took `50381.3072414` seconds while packed profiles produced about `3.57--3.58 GB` `heartbeat.ndjson` files and single-thread profiles about `0.77 GB` each	Report elapsed time only as run metadata and telemetry-load context	Wall-clock time is contaminated by extended heartbeat logging and is not clean model-throughput proof.
Live evidence counts	Checked-in inventory and recount surfaces track proof-surface scale	Describe breadth and maintenance of evidence	Counts are not coverage percentages or pass rates.
Fresh full-suite total	No current checked-in evidence artifact reports one	Decline such a claim explicitly	Absence of a fresh full-suite total is part of the current evidence boundary.
Lightly exercised channels	Several exported channels remain mostly quiescent in the documented matrix	Claim implementation presence and instrumentation	Quiescent counters do not justify strong empirical dominance claims.
Hardware cost and PPA	The repository exposes structural scheduler contours and a pipelined foreground seam, but no checked-in timing/area/PPA measurements	State hardware-cost discussion as an explicit non-claim	Detailed hardware-cost analysis is outside the present evidence envelope.

Table 20: Negative-evidence coverage and current limits.

Case	Expected behavior	Evidence status	Current limitation
Owner mismatch reject	Reject before replay-template reuse, witness fallback, or assist consumption.	Code-backed guard path; weak workload exercise in published matrices.	Not a repository-wide isolation result.
Domain mismatch reject	Reject before cached reuse or cross-domain assist transport is accepted.	Code-backed owner/domain guard; mostly quiescent telemetry.	Not covert-channel or noninterference evidence.
Boundary closed reject	Reject candidate work before replay reuse when serializing or SMT boundary state is closed.	Code-backed boundary guard; lightly exercised in current artifacts.	Not exhaustive serializing-event coverage.
Replay phase mismatch invalidation	Invalidate cached replay state and resume live legality over current bundle state.	Code-backed invalidation vocabulary and replay substrate.	Not global replay determinism.
Structural identity mismatch invalidation	Invalidate replay-template reuse and fall back to current structural witness or reject.	Code-backed template key and structural identity comparison.	Not a partial witness-repair theorem.
Boundary-state mismatch invalidation	Invalidate reuse when stored boundary state no longer matches the live state.	Code-backed template key component.	Not a bundle-wide Acquire/Release theorem.
Stale assist provenance reject	Reject assist work whose donor provenance or freshness assumptions drift before admission.	Code-backed assist provenance and invalidation surfaces plus assistant decision matrix.	Not a fully invisible assist claim.
Accepted assist replay invalidation	Discard an already accepted assist before architectural publication when replay is invalidated.	Test-local lifecycle diagnostic records discard, zero retire records, zero architectural writes, zero committed stores, telemetry events, and carrier publication.	Not end-to-end production retire coverage.
Assist quota reject	Deny assist injection when issue or line budget is unavailable.	Assistant diagnostic observes the expected quota reject counter as `1`.	Not mature assist-throughput evidence.
Assist backpressure reject	Deny assist injection when carrier or resource backpressure blocks reservation.	Assistant diagnostic observes the expected backpressure reject counter as `1`.	Not a broad resource-dominance claim.
Guard negative controls	Reject mismatched owner/context, mismatched domain, closed boundary, invalid replay boundary, and stale witness/template cases.	Safety diagnostic records each expected reject counter as `1` and marks all negative controls passed.	Injected negative controls, not exhaustive security or noninterference coverage.
Stage A success + Stage B late-binding conflict	Report materialization conflict inside the admitted class envelope without reopening legality.	Code-backed path with observed cross-lane conflict counters.	Not generic anonymous port-pressure evidence.
Rollback restores only captured architectural registers / exact bound memory	Restore captured register and exact main-memory ranges, or fail closed if the exact range is unavailable.	Code-backed `ReplayToken` capture/restore surface.	Not backend-global rewind.
Acquire/Release mixed-SMT invalidation not claimed	Do not claim blanket mixed-SMT replay invalidation for every acquire/release interaction.	Explicit manuscript non-claim.	Open verification surface.

Table 21: Checked-in short-run runtime baseline; raw IPC is retired instructions divided by cycles.

Mode	Retired instructions	Cycles	Raw IPC	Last reject kind	Last authority source	Slack reclaim ratio
showcase	169	42	4.0238	cross-lane conflict	structural witness	0.4130
vt	168	40	4.2000	cross-lane conflict	structural witness	0.4255
novt	188	56	3.3571	none	structural witness	0.0000
alu	188	56	3.3571	none	structural witness	0.0000
max	168	40	4.2000	cross-lane conflict	structural witness	0.4255
lk	168	40	4.2000	none	structural witness	0.8158
bnmcz	168	40	4.2000	none	structural witness	0.7867

Table 22: Long-run NativeVLIW SPEC-like diagnostic matrix observed in artifact 20260425_021210_724_matrix at 1,000,000 configured iterations.

Mode	Workload shape	Iter.	Raw IPC	Retire width	Cycles	Retire cycles	Stall share	Lane real.	Wide success
alu	spec-like-single-thread-int	1,000,000	3.3571	3.6154	1,555,555	1,444,443	0.00	1.0000	0.5893
novt	spec-like-single-thread-vector	1,000,000	3.3571	3.6154	1,555,555	1,444,443	0.00	1.0000	0.5893
vt	spec-like-rate-packed-scalar	1,000,000	4.2000	4.6667	5,000,000	4,500,000	0.00	1.0000	0.9000
max	spec-like-rate-packed-mixed	1,000,000	4.2000	4.6667	5,000,000	4,500,000	0.00	1.0000	0.9000
lk	spec-like-latency-hiding-memory	1,000,000	4.2000	4.6667	5,000,000	4,500,000	0.00	1.0000	0.9000
bnmcz	spec-like-bank-rotated-memory	1,000,000	4.2000	4.6667	5,000,000	4,500,000	0.00	1.0000	0.9000

Values are diagnostic counter ratios within this benchmark envelope. Host elapsed time is not used as throughput evidence for this run because extended heartbeat telemetry produced multi-GB heartbeat.ndjson files in packed profiles.

Table 23: Bundle realization counters observed in the 1,000,000-iteration NativeVLIW diagnostic matrix.

Profiles	Width0	Width1	Width2	Width3	Width4	Multi-lane exec.	Cluster choices	Partial width	Lane real.	Drop share
alu, novt	83,333	83,334	166,667	0	916,665	1,166,666	916,665	166,667	1.0000	0.0000
vt, max, lk, bnmcz	375,000	0	0	0	4,375,000	4,375,000	4,500,000	0	1.0000	0.0000

Single-thread profiles retain partial-width observations, while the packed profiles show partial_width=0. In both groups, lane materialization preserved prepared lanes and no width-drop share was observed in these counters.

Table 24: Packed-mode retire-visible surface and interpretation boundary.

Surface	Observed value	Within-envelope reading	Boundary
Physical typed-lane topology	`W=8`	Fixed physical container and typed-lane substrate.	Not a usable eight-slot retire-width claim in this diagnostic workload.
Raw cycle IPC	4.2000	Diagnostic counter ratio over total cycles for packed profiles.	Not a universal performance bound.
Retire-normalized width	4.6667	Observed retire-visible diagnostic width over retire cycles for packed profiles.	Not an architectural maximum theorem.
Effective diagnostic envelope	$W_{\text{eff}} \approx 5$	Descriptive denominator for the observed retire-visible heterogeneous surface.	Used only within this benchmark envelope.

Table 25: Most exercised pressure surfaces in the current `1,000,000`-iteration SPEC-like artifact.

Mode	SMT register-group legality-witness rejects	Class-flexible injects	Hard-pinned injects	NOPs due to no class capacity
`alu`	0	0	0	0
`novt`	0	0	0	0
`vt`	3,375,000	2,500,000	0	0
`max`	3,375,000	2,500,000	0	0

Table 26: Replay phase-pair and replay-template reuse diagnostics in artifact `20260425_021210_724_matrix`.

Diagnostic case	Iterations	Lookup attempts	Hits	Misses	Hit rate	Saved / invalidated / fallback counters	Bounded reading
Stable replay phase	1,000,000	3,000,000	3,000,000	0	100%	18,000,000 checks saved; 3,000,000 invalidations	Three ready checks per replay iteration.
Rotating replay phase	1,000,000	3,000,000	3,000,000	0	100%	18,000,000 checks saved; 3,999,999 invalidations	Same replay-aware cycles as stable; delta 0.
Stable template reuse	–	4,096	4,095	1	99.98%	1 fallback-to-live-witness	Reuse after one template-construction miss.
Phase-key invalidation	–	4,096	0	4,096	0.00%	4,095 phase-key invalidations; 4,096 fallbacks	Phase-key drift closes reuse.
Structural-identity invalidation	–	4,096	0	4,096	0.00%	4,095 structural invalidations; 4,096 fallbacks	Structural drift closes reuse.
Boundary-state invalidation	–	4,096	0	4,096	0.00%	4,095 boundary invalidations; 4,096 fallbacks	Boundary drift closes reuse.
Template aggregate	–	16,384	4,095	12,289	24.99%	12,289 fallbacks; 16,384 witness accesses	Diagnostic counter evidence, not a global replay guarantee.